

# **Metaheuristic-Assisted Deep Learning for Brain Hemorrhage Detection and Classification**

## *A PROJECT REPORT*

*Submitted in partial fulfillment of the requirements for the award of the degree*

*of*

**BACHELOR OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE AND ENGINEERING**

*Submitted by*

|                             |                   |
|-----------------------------|-------------------|
| <b>DOKKARI SOWJANYA</b>     | <b>22A51A0579</b> |
| <b>TANKALA CHARISMA</b>     | <b>22A51A05C4</b> |
| <b>KOMPALLI KEERTHI</b>     | <b>22A51A0595</b> |
| <b>TEETI PRASAD</b>         | <b>22A51A05C5</b> |
| <b>BODNAIK THARUN KUMAR</b> | <b>22A51A0574</b> |

*Under the Guidance of*

**Sri. D. Sreenu Babu**

Asst Professor



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**ADITYA INSTITUTE OF TECHNOLOGY AND MANAGEMENT**

**(An Autonomous Institution)**

**Approved by AICTE, Permanently Affiliated to JNTUGV – Vizianagaram**

**Accredited by NBA & NAAC with A+, Recognized Under 2(f) 12(B) of UGC**

**K.Kotturu, Tekkali, Srikakulam - 532201, Andhra Pradesh.**

**April - 2026**

## DECLARATION



*“We hereby declare that the project entitled **“Metaheuristic-Assisted Deep Learning for Brain Hemorrhage Detection and Classification”** Submitted for the award of the degree of Bachelor of Technology in **Computer Science and Engineering** is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the completion for the award of any other degree, associate ship, fellowship or any other similar titles.*

**PLACE:** Tekkali

**DATE :**

**Dokkari Sowjanya**

**22A51A0579**

**Tankala Charisma**

**22A51A05C4**

**Kompalli Keerthi**

**22A51A0595**

**Teeti Prasad**

**22A51A05C5**

**Bodnaik Tharun Kumar**

**22A51A0574**

# ADITYA INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(An Autonomous Institution)

Department of Computer Science and Engineering



## CERTIFICATE

This is to certify that the project report entitled “**Metaheuristic-Assisted Deep Learning for Brain Hemorrhage Detection and Classification**” being submitted by DOKKARI SOWJANYA (Registered No.:22A51A0579), TANKALA CHARISMA (Registered No.:22A51A05C4), KOMPALLI KEERTHI (Registered No.:22A51A0595), TEETI PRASAD (Registered No.:22A51A05C5), BODNAIK THARUN KUMAR (Registered No.:22A51A0574) In partial fulfillment for the award of the Degree of Bachelor of Technology in **Computer Science and Engineering** to the Jawaharlal Nehru Technological University Gurajada Vizianagaram is a record of Bonafide work carried out under my guidance and supervision.

**Dr. Y. Ramesh** M.Tech, Ph.D.  
Head of the Department  
Computer Science and Engineering

**Sri. D.Sreenu Babu**  
Asst Professor

## ACKNOWLEDGEMENT

---

We wish to thank **D. Sreenu Babu** for his kind support and his valuable suggestions and encouragement helped us a lot in carrying out this project work as well as in bringing this project to this form.

We take this opportunity to express our sincere gratitude to our Director **Prof. V. V. Nageswara Rao** for his encouragement in all respect.

We take the privilege to thank our principal **Dr. A. S. Srinivasa Rao** for his encouragement and support.

We are also very much thankful to **Dr. Y. Ramesh**, Head of the Department, Computer Science and Engineering for his help and valuable support in completing the project

We are also thankful to all staff members in the Department of Computer Science and Engineering, for their feedback in the reviews and kind help throughout our project

Last but not least, we thank all our classmates for their encouragement and help in making this project a success by maintaining consistency in working out the project during from the data collection phase to implementation phase.

It is their help and support, due to which we were able to complete the design and technical report.

|                             |                   |
|-----------------------------|-------------------|
| <b>Dokkari Sowjanya</b>     | <b>22A51A0579</b> |
| <b>Tankala Charisma</b>     | <b>22A51A05C4</b> |
| <b>Kompalli Keerthi</b>     | <b>22A51A0595</b> |
| <b>Teeti Prasad</b>         | <b>22A51A05C5</b> |
| <b>Bodnaik Tharun kumar</b> | <b>22A51A0574</b> |

## Program Outcomes (PO)



1. **ENGINEERING KNOWLEDGE:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **PROBLEM ANALYSIS:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **DESIGN/DEVELOPMENT OF SOLUTIONS:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **CONDUCT INVESTIGATIONS OF COMPLEX PROBLEMS:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **MODERN TOOL USAGE:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **THE ENGINEER AND SOCIETY:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **ENVIRONMENT AND SUSTAINABILITY:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **ETHICS:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **INDIVIDUAL AND TEAM WORK:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **COMMUNICATION:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **PROJECT MANAGEMENT AND FINANCE:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **LIFE-LONG LEARNING:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

### Program Specific Outcomes

---

- **PSO1:-** Apply mathematical foundations, algorithmic principles, and theoretical computer science in the modeling and design of computer-based systems in a way that demonstrates comprehension of the tradeoffs involved in design choices.
- **PSO2:-** Demonstrate understanding of the principles and working of the hardware and software aspects of computer systems.
- **PSO3:-** Use knowledge in various domains to identify research gaps and hence to provide solution to new ideas and innovations

### PO-PSO Mapping

---

| PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 | PSO1 | PSO2 | PSO3 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|------|
| 3   | 3   | 3   | 3   | 3   | 2   | 2   | 1   | 3   | 2    | 3    | 3    | 3    | 3    | 2    |

## Project Details

---

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Batch No :</b>       | 17                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Project Title :</b>  | Metaheuristic-Assisted Deep Learning for Brain Hemorrhage Detection and Classification                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Area /Domain :</b>   | Deep Learning                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Application :</b>    | Health Care                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Description :</b>    | Brain Hemorrhage is a medical condition that requires immediate medical attention and is caused by bleeding in the brain. It is essential to detect brain hemorrhage in time to avoid serious consequences, such as death. The proposed idea for the project is to implement a Metaheuristic-Assisted Deep Learning system that helps in the detection and classification of brain hemorrhage in patients based on their images taken using a CT scan. A Convolutional Neural Network (CNN) is employed to automatically detect and classify brain hemorrhage in patients. The parameters of the network are optimized using a metaheuristic algorithm like Particle Swarm Optimization (PSO) to enhance the performance of the network. |
| <b>Project Outcome:</b> | This project aims to create a Metaheuristic-Assisted Deep Learning approach that enhances the detection and classification of brain hemorrhage based on CT scan images. The approach uses Convolutional Neural Networks and Particle Swarm Optimization techniques to improve the efficiency of the model. It is useful in the early detection of brain hemorrhage, which might be difficult to diagnose at the time. It is useful as it assists doctors with faster and more accurate results.                                                                                                                                                                                                                                          |

## Batch Details

---



### **Project Guide**

**Sri. D.Sreenu Babu**

Asst Professor

Dept of Computer Science and Engineering

Contact No: +91 9030614556

[babu539@gmail.com](mailto:babu539@gmail.com)



**Dokkari Sowjanya**

Regd No. : 22A51A0579

[sowjanyadokkari@gmail.com](mailto:sowjanyadokkari@gmail.com)

Contact No: +91 8328621541



**Tankala Charisma**

Regd No : 22A51A0579

[charismatankala@gmail.com](mailto:charismatankala@gmail.com)

Contact No: +91 9392858095



**Kompalli Keerthi**

Regd No. : 22A51A0595

[Keerthikompalli890@gmail.com](mailto:Keerthikompalli890@gmail.com)

Contact No: +91 6301706452



**Teeti Prasad**

Regd No. : 22A55A0517

[prasadteeti9@gmail.com](mailto:prasadteeti9@gmail.com)

Contact No: +91 7997296117



**Bodnaik Tharun Kumar**

Regd No.:22A51A0574

[Tharunkumarbad3032@gmail.com](mailto:Tharunkumarbad3032@gmail.com)

Contact No.:+91 7013102302



## ABSTRACT

---

Brain hemorrhage is a life-threatening condition for the nervous system and requires careful diagnosis. If not diagnosed properly, it can result in high mortality rates and complications. In recent years, improvements in deep learning have provided new methods for effectively analyzing medical images. While Convolutional Neural Networks (CNNs) can perform medical analysis well, their effectiveness relies heavily on finely tuned parameters. This research project aims to create and apply an optimal deep learning solution for automatically classifying brain hemorrhage from medical images using a CNN and Particle Swarm Optimization (PSO). The study uses a CNN to analyze brain medical images for proper identification and classification. Furthermore, PSO can effectively identify and determine key parameters for deep learning algorithms. Extensive experimental investigations have proven the efficacy and effectiveness of the CNN-PSO framework, as it reaches high levels of classification accuracy, robustness, and dependability when it comes to the classification of various categories related to cases of brain hemorrhage. The experiments have also justified the significant improvement that can be ensured if Particle Swarm Optimization approaches are combined with Deep Learning algorithms for diagnosis purposes.

**Keywords:** CNN, Brain hemorrhage, Deep Learning, PSO algorithm

# TABLE OF CONTENTS

|                  | <b>Contents</b>                   | <b>Page. No</b> |
|------------------|-----------------------------------|-----------------|
|                  | Candidate's Declaration           | ii              |
|                  | Supervisor's Certificate          | iii             |
|                  | Acknowledgements                  | iv              |
|                  | PO – PSO mapping                  | v               |
|                  | Project Details                   | vii             |
|                  | Batch Details                     | viii            |
|                  | Abstract                          | ix              |
|                  | Table of Contents                 | x               |
|                  | List of Tables                    | xii             |
|                  | List of Figures                   | xiii            |
|                  | List of Abbreviations             | xiv             |
| <b>Chapter 1</b> | <b>Introduction</b>               | <b>1-5</b>      |
| 1.1              | Introduction of the project       | 1               |
| 1.2              | General Concepts                  | 2               |
| 1.3              | Motivation                        | 2               |
| 1.4              | Problem Statement                 | 3               |
| 1.5              | Objective                         | 3               |
| 1.6              | Overview                          | 4               |
| 1.7              | Scope                             | 4               |
| <b>Chapter 2</b> | <b>Literature Survey</b>          | <b>6-10</b>     |
| <b>Chapter 3</b> | <b>Methodology</b>                | <b>11-20</b>    |
| 3.1              | Existing System                   | 11              |
| 3.2              | Proposed methodology              | 12              |
| 3.3              | Dataset                           | 14              |
| 3.4              | Model details                     | 16              |
| <b>Chapter 4</b> | <b>Requirement Specifications</b> | <b>21-25</b>    |
| 4.1              | Functional Requirements           | 21              |

|                  |                                                |              |
|------------------|------------------------------------------------|--------------|
| 4.2              | Non-Functional Requirements                    | 21           |
| 4.3              | Hardware requirements                          | 23           |
| 4.4              | Software requirements                          | 23           |
| 4.5              | Required Python Libraries                      | 23           |
| <b>Chapter 5</b> | <b>Code</b>                                    | <b>26-36</b> |
| <b>Chapter 6</b> | <b>Performance metrics and result analysis</b> | <b>37-46</b> |
| 6.1              | Performance metrics                            | 37           |
| 6.2              | Result analysis                                | 40           |
| <b>Chapter 7</b> | <b>Conclusion</b>                              | <b>47</b>    |
| <b>Chapter 8</b> | <b>Future scope</b>                            | <b>48</b>    |
|                  | <b>References</b>                              | <b>49-50</b> |
|                  | List of Enclosures                             | 51-52        |
|                  | Paper Acceptance                               | 53           |
|                  | Research Paper                                 | 54-58        |

## LIST OF FIGURES



| Figure | Title                                              | Page |
|--------|----------------------------------------------------|------|
| 3.1    | Methodology                                        | 13   |
| 3.2    | MobileNetV2 for Brain CT Scan Image Classification | 17   |
| 3.3    | CNN+PSO architecture                               | 18   |
| 6.1    | Confusion matrix of the model                      | 42   |
| 6.2    | CNN accuracy curve                                 | 43   |
| 6.3    | CNN loss curve                                     | 43   |
| 6.4    | CNN+PSO accuracy curve                             | 44   |
| 6.5    | CNN+PSO loss curve                                 | 44   |
| 6.6    | Class-wise accuracy bar plot                       | 45   |
| 6.7    | Model comparison bar plot                          | 46   |

## LIST OF TABLES



| Figure | Title                                                            | Page |
|--------|------------------------------------------------------------------|------|
| 3.1    | Dataset sample images                                            | 15   |
| 6.1    | Confusion matrix representation                                  | 38   |
| 6.2    | Comparative analysis of different models                         | 41   |
| 6.3    | Comparative analysis of per-class accuracies of different models | 41   |

## LIST OF ABBREVIATIONS



|      |                                   |
|------|-----------------------------------|
| CNN  | Convolutional Neural Network      |
| PSO  | Particle Swarm Optimization       |
| CT   | Computed Tomography               |
| AI   | Artificial Intelligence           |
| DL   | Deep Learning                     |
| ROC  | Receiver Operating Characteristic |
| AUC  | Area Under Curve                  |
| MRI  | Magnetic Resonance Imaging        |
| ReLU | Rectified Linear Unit             |

# CHAPTER - 1

---

## INTRODUCTION

### 1.1 Introduction of the Project

Brain disorders affect brain-related functions involving cognition, movement, speech, and memories. Another form of brain disorder is brain hemorrhage, which is a severe form of a disorder affecting the brain. It is usually caused by the bursting of blood vessels, resulting in bleeding thus leading to a rise in intracranial pressure and damage to neurons in the brain. Symptoms include a severe headache, weakness, difficulties in speech, or loss of consciousness.

Due to this fact, deep learning approaches are widely employed for efficient processing of vast biomedical data generated by IoMT technologies, since they automatically extract complex features and allow for fast analysis, thus enabling accurate, real-time clinical decision-making. Complex image features can also be learned automatically by deep learning models without human intervention for feature engineering. Hierarchical image features extracted by CNNs can identify subtle image patterns corresponding to hemorrhage that can be used to successfully identify hemorrhage in CT scans. Among various deep learning models, CNN-based models have shown high performance for spatial feature extraction as well as the identification of CT image slices as hemorrhage or non-hemorrhage to enhance the reliability of the process to identify hemorrhage.

With deep learning algorithms, intricate features of biomedical images can be learned by computers automatically without the need for human intervention in feature engineering. Convolutional Neural Networks (CNNs) pick out hierarchical image features that assist in identifying minute differences related to hemorrhage in CT scans to accurately detect hemorrhage in the brain. Within deep learning algorithms, high accuracy is achieved by CNN-based approaches to pick out spatial features as well as classify the CT scan into either hemorrhage or non-hemorrhage categories to increase.

Particle Swarm Optimization (PSO) is a metaheuristic technique that is triggered by the social behavior of swarms and is known to work efficiently for hyperparameter optimization in deep learning networks. In medical imagery problems, studies using PSO have been found to optimize deep learning CNN parameters automatically, thus improving performance as opposed to doing the optimization manually.

## 1.2 General Concepts

Brain hemorrhage is a serious medical condition whereby there is a leakage of blood in the brain as a result of the bursting of a blood vessel in the brain. This is a medical condition that may lead to serious health hazards, whereby there may be damage to the brain or the brain functions, or even death if the condition is not identified and treated at the initial stages. A CT scan is a medical imaging technique that is mostly used by medical practitioners to diagnose the condition of brain hemorrhage, whereby the doctor is able to obtain a clear image of the structure of the brain and the bleeding occurring in the brain.

In recent years, Artificial Intelligence (AI) and Deep Learning techniques have been identified as having tremendous potential for the application of medical images. Deep learning techniques, especially Convolutional Neural Networks (CNNs), have the ability to obtain the most crucial features of the images and identify the hidden patterns of the images.

The objective of this project is to design a system based on the concept of ‘Metaheuristic-Assisted Deep Learning’ that will be used to identify and classify brain hemorrhage. In this project, a CNN model will be used to scan the images that will be obtained from the CT scan. In addition to this, a metaheuristic optimization algorithm, such as PSO, will be used to optimize the parameters of the CNN model to improve the performance of the model. The proposed system will assist the doctors in making decisions efficiently, accurately, and reliably.

## 1.3 Motivation

Brain hemorrhage is an important clinical issue where a blood vessel in the brain bursts to cause bleeding within or around brain tissue. Brain damage, neurological dysfunction, and even death might arise as complications when the brain hemorrhage disease remains untreated due to its failure to diagnose in an earlier period. For these reasons, an effective way of diagnosing brain hemorrhage is highly essential. But the analysis of a huge collection of CT scans of the brain requires too much time. At the early stages, it becomes hard to detect the presence of brain hemorrhage due to the symptoms being mild.

As a result of the rapid development of AI technology, Deep Learning, and the processing of images, tremendous developments have been experienced in the field of medicine. There are numerous deep learning approaches that have proven to be capable of extracting meaningful information from medical images with high precision, with CNN being the leading one among these approaches. It is possible for such technologies to save human labor, prevent mistakes, and ensure that the results are consistent. Hence, a lot of efforts have been geared towards making an intelligent system to help diagnose critical illnesses such as brain hemorrhage.



The purpose of this project is to design a smart and automatic system that will be able to detect brain hemorrhage using CT scan images through deep learning. In addition, by incorporating optimization techniques such as Particle Swarm Optimization (PSO), it will enable the selection of optimal parameters to improve the accuracy of the system. The advantage of this approach lies in its ability to significantly reduce the burden placed on doctors and to accelerate the diagnosis process, which makes it especially valuable during emergencies.

## **1.4 Problem Statement**

Brain hemorrhage is one of the most dangerous conditions that should be detected in its early stages to avoid fatal consequences like brain damage or even death. However, there are some limitations connected with the manual examination of CT scans performed by experienced radiologists. In this case, the amount of time required to conduct this procedure increases with the increase in the number of CT scans being examined. Besides, the manual identification of brain hemorrhage can take a lot of time due to the inability to distinguish between major and minor forms of hemorrhage. Therefore, it seems necessary to apply a more effective method of CT scan examination.

In order to overcome such difficulties, it is essential to design a system that will be capable of analyzing and classifying the occurrence of brain hemorrhage by using CT-scan images through the use of advanced technologies. There is the capacity by deep learning methods, especially the application of CNN in analyzing brain images. On the other hand, there are optimization techniques that can improve the efficiency and effectiveness of the process through the application of PSO. The problem is thus the development of a system that will not only be accurate in its analysis but will reduce diagnosis time and prevent errors.

## **1.5 Overview**

Deep Learning algorithms have experienced an explosive development that has greatly impacted the realm of medical image processing, especially in the identification of vital diseases such as brain hemorrhages. Brain hemorrhages represent a critical and lethal health problem that necessitates an efficient and precise diagnosis, usually carried out via CT scan imaging. Conventional processes rely greatly on the skills of radiologists, making them extremely time-consuming and susceptible to errors, particularly during emergencies. With the increasing amount of imaging data, there is an urgent need for computer-assisted systems to facilitate accurate diagnoses.

For this reason, deep learning-based algorithms, notably CNNs, have become extensively utilized in medical imaging tasks. The CNN algorithm is very efficient in identifying complex features in images and conducting precise classification while not requiring manual feature extraction. Other complex CNN structures, including

VGG16 and ResNet50, have been implemented to increase accuracy levels, although problems such as overfitting, heavy computation costs, and parameter optimization difficulties still persist.

The suggested methodology for brain hemorrhage detection in this project employs a hybrid approach by using the CNN technique together with the Particle Swarm Optimization (PSO). In addition to using CNN for feature extraction and classification tasks, PSO is implemented in this project to fine-tune several key parameters including learning rate, number of neurons, and dropout rate. The use of such an approach enables us to reach higher accuracy levels, minimize error, and improve our model's generalization capabilities.

## **1.6 Objective**

The main aim behind the design of this system is to design a powerful and efficient deep learning-based model for detecting brain hemorrhage using CT scan images. The model proposed in this paper makes use of the convolutional neural network (CNN), which has the ability to automatically learn and extract relevant features from medical images for proper classification. In order to ensure that patients receive timely and appropriate treatment, there is a need for early detection of brain hemorrhage. Through the application of deep learning, the dependency on human effort has been minimized.

Apart from feature extraction and classification, Particle Swarm Optimization (PSO) is used in the model to tune important hyperparameters like learning rate, neuron count, and dropout rate. Tuning will contribute towards ensuring that the model performs optimally in terms of generalization. Another goal for this project is to attain high values for metrics like accuracy, precision, recall, and F1 score. Overall, the goal is to develop an efficient, automated, and reliable system that assists medical experts to make accurate and timely diagnoses.

## **1.7 Scope**

The scope of this study includes implementation of highly sophisticated deep learning methods combined with an optimization technique in the field of medicine, more precisely – brain hemorrhage identification based on CT scan pictures. Indeed, the problem of brain hemorrhage is rather complicated; in order to ensure effective patient treatment, it must be diagnosed timely and accurately enough to avoid negative consequences and save the person's life. In this context, the suggested model combining Convolutional Neural Networks and Particle Swarm Optimization seeks to provide additional assistance to the radiologist during the analysis of images and classification. By means of automation of the detection process, human error is decreased as well as the possibility to make timely decisions due to the reduced time frame necessary for the process. Therefore, this study will bring considerable contribution to improving classification accuracy and optimizing the operation of the algorithm.

Apart from detecting brain hemorrhages, the scope of the project can be expanded to include other medical imaging applications such as tumor detection, stroke analysis, and other neurological diseases. The adaptability of deep learning algorithms makes them capable of being trained with various types of datasets, giving the model the ability to serve multiple purposes. Moreover, the accuracy and effectiveness of the proposed model can be improved through the use of bigger and varied datasets, making it better at generalization and more reliable during real-time clinical data processing. Some of the future improvements that could be implemented include implementing the model into real-world situations, integration with hospital information systems, and the development of an easy-to-use interface for medical professionals.

## CHAPTER - 2

### LITERATURE SURVEY

This chapter is dedicated to discussing some previous researches on brain hemorrhage detection through deep learning algorithms. Mostly, the Convolutional Neural Network (CNN) method is considered for studying medical imaging data, particularly from the CT scan for effective and timely detection of brain hemorrhages. In addition, many researchers have utilized some sophisticated deep learning algorithms as well as hybrid deep learning algorithms for brain hemorrhage detection.

Nonetheless, the limitations of these models include the high computation complexity, overfitting problem, and the challenge of determining optimum hyperparameters. As such, an attempt is made to combine the CNN model with an optimization algorithm like Particle Swarm Optimization (PSO) to improve its performance and effectiveness. The literature review plays a key role in revealing the research gap and necessitating the proposed CNN-PSO deep learning technique for brain hemorrhage detection.

#### 2.1 Literature Survey

Kang et al. [1] (2025) devised a deep learning-enabled system for the detection of intracranial hemorrhage from CT scan images. This work relied on state-of-the-art convolutional neural networks that could automatically recognize hemorrhagic areas, thereby increase the accuracy of diagnosis and decrease the involvement of radiologists. It had remarkable performance when applied to classification tasks; however, because of the dependence on annotated datasets and high computation requirements, it was not applicable to resource-limited healthcare environments.

Chaudhary et al. [2] (2025) presented a multiclass intracranial hemorrhage detection model based on deep ensemble learning algorithms. This method had the capacity to detect multiple types of hemorrhages and employed a confidence-based triage to facilitate clinical decisions. Despite the good performance and robustness of their approach, the increased complexity due to the ensemble learning model decreased its potential use in urgent situations.

A study carried out by Kumar et al. [3] (2025) came up with an artificial intelligence-based model capable of detecting and classifying hemorrhage from CT scans. They specifically concentrated on detecting hemorrhages at an early stage since early detection helps in timely treatment. Though the model performed

well, it is still difficult for the model to be generalized since there exist significant differences between patients.

Reethika et al. [4] (2025) designed a deep learning ensemble technique to combine different models and hence increase the accuracy of hemorrhage detection. Using the method significantly reduces the biases of individual models. However, the drawback is that the computational costs are higher because of using several models.

Li et al. [5] (2025) introduced a mixed deep learning framework by combining various neural network structures that can extract both local and global features from CT scan data. This framework demonstrated better accuracy in detecting abnormal conditions than using only one neural network model. Although the method is efficient, it results in a more complex architecture that needs proper parameter configuration.

Sharma et al. [6] (2025) introduced an effective CNN structure that enables real-time identification of intracranial bleeding. The lightweight framework is efficient in terms of computational speed and memory usage. However, the simplicity of the network may affect its accuracy when compared to other complex networks.

A study by Zhang et al. [7] (2025) demonstrated a transformer-based model to classify intracranial hemorrhages from CT scan data. The use of an attention mechanism allowed the model to extract spatial information from the images. Despite its effectiveness, it is important to note that transformer models demand substantial computational resources.

Ahmed et al. [8] (2024) performed a comparative study between machine learning and deep learning approaches in CT images of brain hemorrhage detection. In their study, several models were considered and analyzed, showing that deep learning is superior to conventional learning methods in terms of prediction capabilities and robustness. But their study lacked real-time performance and large labeled datasets as well.

D'Angelo et al. [9] (2024) proposed a deep learning approach to detect intracranial hemorrhage with an emphasis on high accuracy and efficient computation times. As a result, their approach managed to decrease the time required to obtain results while preserving high performance. Although efficient, their method requires good quality images as inputs for accurate analysis.

Tapia et al. [10] (2024) provided a study on the comparison between different YOLO models used for hemorrhage detection on CT data. Their emphasis was on real-time performance and showed promising

results regarding the potential applications of object detection algorithms in medical field. Unfortunately, some YOLO models might suffer from sensitivity issues when detecting small hemorrhages.

Gencturk et al. [11] (2024) suggested a deep learning algorithm that can detect and segment subdural hemorrhage from CT images. This model offered an accurate localization that can be helpful for better understanding. On the contrary, such segmentation models require pixel-wise labeling, which takes much time.

Pratama et al. [12] (2024) used meta-heuristic optimization algorithms to tune CNN hyperparameters such as learning rate and number of neurons. This method was quite effective and allowed reaching more efficient parameter configuration. Nevertheless, such tuning procedure is rather resource-consuming.

Kar et al. [13] (2024) designed an automated tool based on deep learning that could detect intracranial hemorrhages in medical images. This model showed high accuracy and allowed decreasing dependence on expert opinion. However, this model may have poor performance with unseen datasets due to domain differences.

Moazedi et al. [14] (2024) examined various preprocessing methods alongside deep learning algorithms in hemorrhage detection. The study emphasized the significance of preprocessing in enhancing the accuracy of models. Nonetheless, determining the most suitable preprocessing method is complex and highly dependent on the dataset used.

Chen et al. [15] (2024) suggested an attention-based deep neural network algorithm for the detection of intracranial hemorrhages. The network paid particular attention to significant areas in CT scans for improved feature detection and classification. Attention-based algorithms are, however, relatively complex and computationally expensive.

Hassan et al. [16] (2024) presented an ensemble deep learning framework aimed at increasing the robustness and generalization of hemorrhage detection algorithms. Ensemble methods involved training multiple models to enhance prediction results. Nonetheless, ensemble learning methods demand high processing resources.

Singh et al. [17] (2024) performed a comparative analysis of various deep learning architectures for hemorrhage detection. Their study provided insights into the strengths and weaknesses of different models. However, the analysis is limited to specific datasets and may not generalize to all clinical scenarios.

Verma et al. [18] (2023) proposed a CNN-based method for automated intracranial hemorrhage detection using CT scan images. Their model demonstrated reliable classification performance and reduced manual effort. However, it requires large labeled datasets for effective training.

Gupta et al. [19] (2023) applied transfer learning techniques to improve hemorrhage classification accuracy. By leveraging pre-trained models, their approach achieved better performance with limited data. However, transfer learning models may suffer from domain mismatch when applied to medical datasets.

Ali et al. [20] (2023) developed an efficient deep learning model for early detection of intracranial hemorrhage. Their method focused on improving sensitivity to detect early-stage abnormalities. However, the model may produce false positives, affecting reliability.

Tanwar et al. [21] (2023) proposed a CNN-based hemorrhage detection system integrated into intelligent healthcare environments. Their approach supports automated diagnosis and improves clinical workflow. However, integration with real-world systems requires additional infrastructure and validation.

Wang et al. [22] (2023) validated an AI-based hemorrhage detection tool using real-world clinical data. Their study demonstrated the practical feasibility of deploying AI in healthcare settings. However, regulatory and ethical considerations remain challenges for large-scale adoption.

Arman et al. [23] (2023) combined deep learning with Bayesian optimization for improved hemorrhage classification. Their approach enhanced model performance by optimizing hyperparameters efficiently. However, the optimization process adds computational overhead.

Mahjoubi et al. [24] (2023) developed a CNN-based framework for detecting and classifying cerebral hemorrhage in CT scans. Their model achieved good accuracy across multiple datasets. However, its performance depends heavily on dataset quality and preprocessing techniques.

Badriyah et al. [25] (2023) proposed an enhanced CNN architecture for intracranial hemorrhage prediction. Their model improved feature extraction capabilities and classification accuracy. However, increased architectural complexity may affect training efficiency.

Balipa and Kundapur [26] (2023) explored deep learning combined with optimization techniques for hemorrhage detection. Their approach improved classification performance through parameter tuning. However, optimization increases computational cost and training time.

Ozaltin et al. [27] (2023) discussed AI-based methods for brain hemorrhage detection in healthcare systems. Their work highlighted the importance of intelligent automation in diagnosis. However, the study is more conceptual and lacks detailed experimental validation.

Abdulkader et al. [28] (2023) introduced a publicly available CT hemorrhage dataset for research purposes. This dataset supports training and evaluation of deep learning models. However, dataset limitations such as class imbalance can affect model performance.

Hussain et al. [29] (2022) proposed an attention-based ResNet model for intracranial hemorrhage detection. Their approach improved feature representation and classification accuracy. However, deeper networks require more computational resources and longer training time.

Lee et al. [30] (2020) developed a novel deep learning algorithm for detecting and classifying intracranial hemorrhage from CT images. Their work demonstrated the effectiveness of deep learning in medical imaging. However, earlier models lacked the efficiency and robustness of recent architectures.



## CHAPTER - 3

### METHODOLOGY

The proposed Brain Hemorrhage Detection System will utilize a structured approach of deep learning for the detection of brain hemorrhage. This process starts by collecting images of brain CT scans from publicly available datasets, e.g., the Brain CT Hemorrhage dataset. These images then go through a series of preprocessing operations, e.g., resizing, normalization, and basic augmentation, to improve the quality and diversity of the images.

The Convolutional Neural Network (CNN) model will be utilized to automatically identify the key features of the images of the brain CT scans and classify them accordingly. In order to improve the accuracy and reliability of the proposed model, the proposed model will utilize a Particle Swarm Optimization (PSO) algorithm to optimize the hyperparameters of the model, e.g., the learning rate and the weights. This proposed model will improve the accuracy, reliability, and efficiency of the brain hemorrhage detection process.

#### 3.1 Existing System

The current method for the diagnosis of brain hemorrhage involves the use of techniques such as the manual analysis of brain image scans from the computerized tomography (CT). In this method, radiologists analyze the images to detect the presence of brain hemorrhage, as well as the nature and the degree of the disease. Apart from the manual analysis method, computer aided diagnostic (CAD) tools are sometimes applied to aid in diagnosing brain hemorrhage. Such systems help the expert identify suspicious regions of the image; however, the final diagnosis is left for the experts to make.

However, there are numerous weaknesses associated with these methods. For example, using the manual image analysis technique is usually a tedious procedure, which becomes even more complicated when one has many patients. In addition, the accuracy of the diagnosis usually varies based on the skills of the expert doing the work. Sometimes, the presence of certain human factors such as fatigue and bias can make the diagnosis inaccurate.

In some of the recent studies, the use of Deep Learning methods such as CNN and Object detection algorithms such as YOLO has been employed for automating brain hemorrhage detection. These techniques work by analyzing the images obtained through CT scans and try to classify whether an image is having hemorrhage or not. Although these techniques have proven to be quite effective, they come with their own drawbacks such as lack of efficiency, problems of overfitting, and requirement of large datasets.

## **Disadvantages of the Existing System:**

### **a. Manual diagnosis is time-consuming, subjective, and highly dependent on the Time-Consuming Diagnosis:**

Manual diagnosis of images of the CT scans by radiologists may consume a considerable amount of time, especially when the number of images to be examined is large.

### **b. Possibility of Human Error:**

There might be a possibility of human error, as the presence of early-stage or minor hemorrhages might be overlooked, and incorrect diagnosis might occur.

### **c. Dependence on Expert Radiologists:**

This process depends significantly on the availability of skilled and expert medical professionals.

### **d. Limited Accuracy of Deep Learning Models:**

There might be a limitation of the accuracy of the results obtained by using some of the existing deep learning models, as they might not perform well for the detection of rare types of hemorrhages.

### **e. High Computational Requirements:**

Some of the advanced models might demand a large amount of data and computational power for testing and training.

### **f. Lack of Optimization Techniques:**

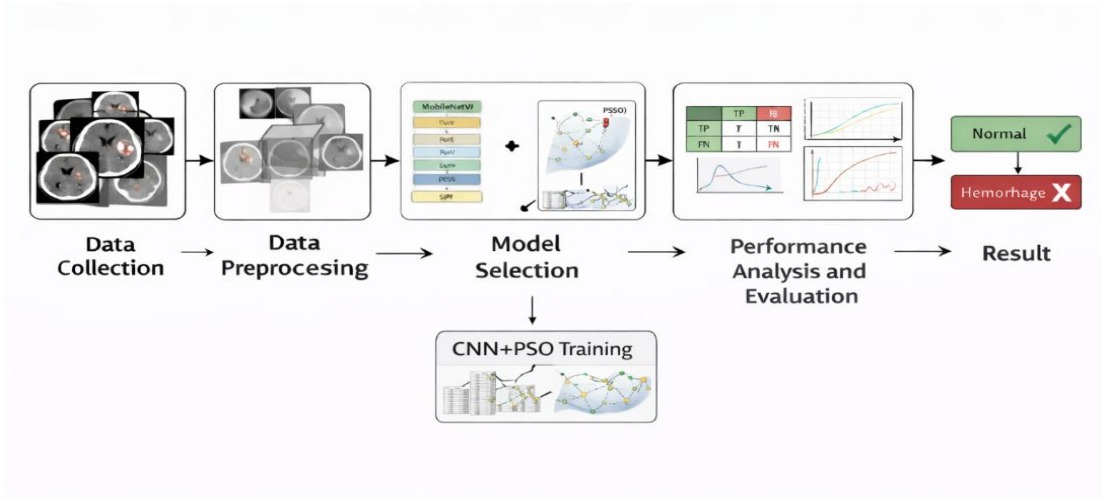
Most of the existing systems might lack the use of optimization techniques.

## **3.2 Proposed methodology**

The proposed system will utilize the Metaheuristic-Assisted Deep Learning technique for the precise detection and classification of brain hemorrhage using the images obtained from the CT scan. The process starts by collecting the images of the brain CT scans from the Brain CT Hemorrhage dataset. These images are then preprocessed using various techniques to ensure the images are of good quality and the dataset is large enough to aid the training process.

Later, a Convolutional Neural Network (CNN) model will be utilized to automatically extract the important features of the images of the brain CT scans and classify the different types of brain hemorrhage. This model helps to identify complex patterns present in the images of the brain CT scans, which might not be easily identifiable.

For better performance of the CNN model, a Particle Swarm Optimization (PSO) technique is used to optimize crucial parameters such as the learning rate and network weights. This will enable the model to obtain maximum accuracy and efficiency in its performance. After optimization of the model, it will be trained and tested on the dataset to identify and classify brain hemorrhage from images obtained from a CT scan.



**Figure 3.1: Proposed Framework for Brain CT Scan Classification Using CNN and PSO**

Figure 3.1 shows how the overall flow process is implemented in the classification of brain CT scan images based on the proposed hybrid CNN and PSO model. The flow process begins with the collection of CT brain scans from relevant data sources. The collected CT scans are processed through the data pre-processing phase, which involves procedures like resizing, normalization, denoising, and image enhancement to increase the quality of the images. Then, the pre-processed images are sent to the modeling phase, where a deep learning model like the CNN will be selected.

Next, the CNN model will take in the CT scan images and automatically extract discriminative and important features from the input data set. Through the implementation of activation functions like ReLU (Rectified Linear Unit), it becomes easier to add non-linearity and make it easy for the model to learn better. For further improvement in the effectiveness of the CNN model, it is essential to incorporate Particle Swarm Optimization (PSO) into the system. The application of PSO helps in optimizing various hyperparameters of the CNN model, including learning rate, neuron counts, and training parameters to achieve lower losses and higher accuracy in the CNN model.

Lastly, the optimized model is implemented and tested on the dataset for detecting and classifying brain hemorrhages. The output generated by the model is classified into two categories, namely, 'Normal' and 'Hemorrhage'. The performance of the developed model can be measured using different performance

metrics, such as accuracy, precision, recall, and F1-score. These performance metrics measure the reliability and efficiency of the proposed system. The objective of this proposed technique is to assist medical experts in making informed decisions by providing reliable and fast results.

## **Advantages of the Proposed System**

### **a. Improved Detection Accuracy:**

The application of Convolutional Neural Networks (CNN) and Particle Swarm Optimization (PSO) helps improve the accuracy of brain hemorrhage detection.

### **b. Automatic Feature Extraction:**

CNN helps automatically identify the important features in the images of the CT scan.

### **c. Early Detection:**

This system helps in the early detection of brain hemorrhage, which in turn helps in the early treatment of the condition.

### **d. Optimized Model Performance:**

The application of PSO helps optimize the performance of the model, making it efficient.

### **e. Faster Diagnosis:**

The system helps in the automated diagnosis of the condition, reducing the time required for the diagnosis compared to manual examination.

### **f. Help for Medical Professionals:**

This system helps doctors make better and faster decisions by acting as a decision-support system.

## **3.3 Dataset Description**

### **3.3.1 Source of Dataset**

In this study, the dataset used for this research was obtained from a publicly available source kaggle, which is the Brain CT Hemorrhage Dataset. The dataset used for this research contains CT scans that are originally obtained from the Near East Hospital in Cyprus for research purposes. Collecting medical imaging data for brain hemorrhage detection can be quite challenging due to privacy concerns and limited access to annotated data. However, this dataset offers a considerable number of CT scan images that are enough for the development and training of deep learning algorithms for brain hemorrhage detection.

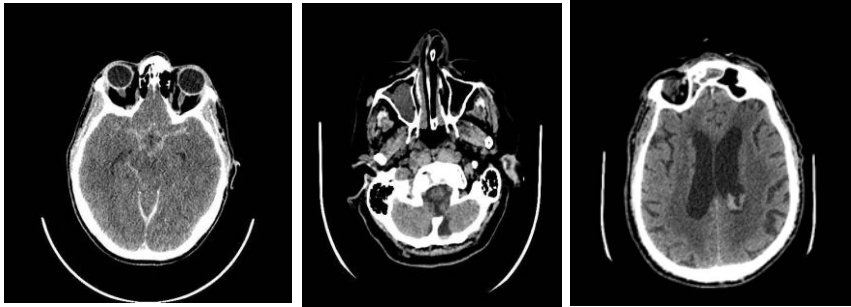
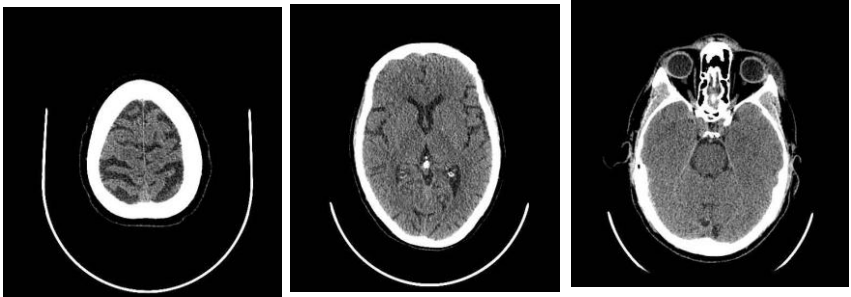
### **3.3.2 Dataset details**

In this study, the dataset used in this particular project includes brain CT scan images, both normal and brain hemorrhage cases. These images are different cross-section images of the human brain, captured using Computed Tomography (CT) scans. CT scans are often employed in medical diagnostics due to their ability

to provide an in-depth visual representation of brain tissues, thus facilitating the detection of abnormalities, such as brain hemorrhage, swelling, and even brain tissue damage.

The dataset includes images that are representative of different brain hemorrhage cases, such as epidural, subdural, intracerebral, subarachnoid, and intraventricular brain hemorrhage. Each of these images, captured using a CT scan, represents different slices of the human brain, thus enabling the deep learning model to learn and differentiate between normal brain tissues and brain hemorrhage cases. The variations in the images enable the model to detect and classify different brain hemorrhage cases more accurately.

**Table 3.1 Dataset sample images**

| Class      | Images                                                                               |
|------------|--------------------------------------------------------------------------------------|
| Hemorrhage |   |
| Normal     |  |

Before using the dataset for model training, the images go through various data preprocessing activities, including resizing, normalization, noise reduction, and enhancement. These activities ensure the images are of the best quality for the model to effectively extract features from the images. After preprocessing, the images are split into various sets to ensure proper evaluation of the system.

The availability of various CT scan images and the different types of hemorrhage enable the model to effectively learn complex features of the images, particularly the bleeding of the brain. This ensures the reliability of the proposed system for the detection of hemorrhage using the images from the CT scans.

On the whole, the dataset is found to have sufficient medical imaging data that is useful for the development of an automated system that detects and classifies brain hemorrhage, thereby helping medical professionals diagnose the condition more accurately.

### **3.4 Model Details**

#### **3.4.1 Convolutional Neural Network (CNN)**

Convolutional Neural Networks (CNNs) is utilized as a major deep learning model to detect and classify brain hemorrhage in images obtained through a CT scan. CNN is a commonly utilized model in image processing due to its ability to automatically extract essential features from an image without having to manually program them.

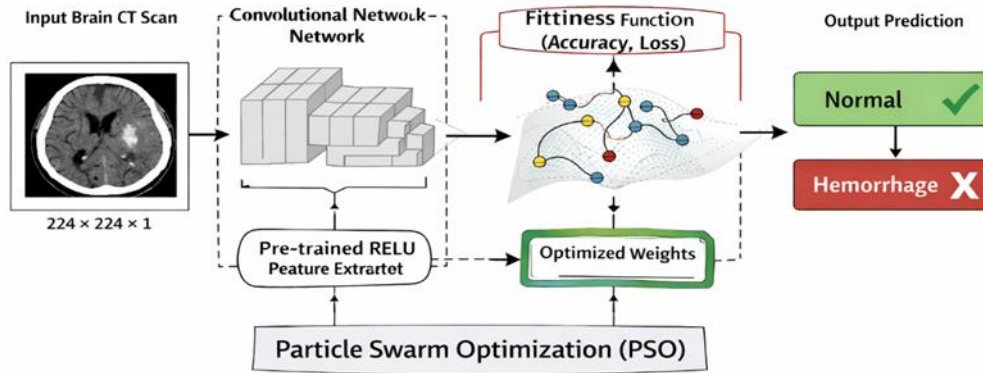
The CNN model has several layers, including convolutional layers, pooling layers, and fully connected layers. Convolutional layers are responsible for automatically learning essential features such as edges, textures, and patterns in images obtained through a CT scan. This is achieved through the application of filters to an image in order to identify features related to brain hemorrhage.

After feature extraction, pooling layers are applied to reduce the spatial dimensions of the feature maps obtained in the previous step, which also reduces the complexity of the model. Pooling also helps in retaining the most important information obtained from the features extracted.

Finally, the features extracted are fed into fully connected layers, which are responsible for performing the classification task. The final layer uses an activation function such as Softmax or Sigmoid to classify the CT scan images into normal brain images or hemorrhage types.

#### **3.4.2 MobileNetV2**

MobileNetV2 is a Convolutional Neural Network (CNN) architecture that is efficient and lightweight, making it a preferred choice for image classification and other computer vision tasks. It is a creation of researchers at Google, and it has been extensively used in deep learning scenarios where efficiency and high performance are necessary. This architecture is appropriate for use in medical image analysis due to its ability to extract key features from images while at the same time being efficient in terms of parameters compared to other deep learning models.

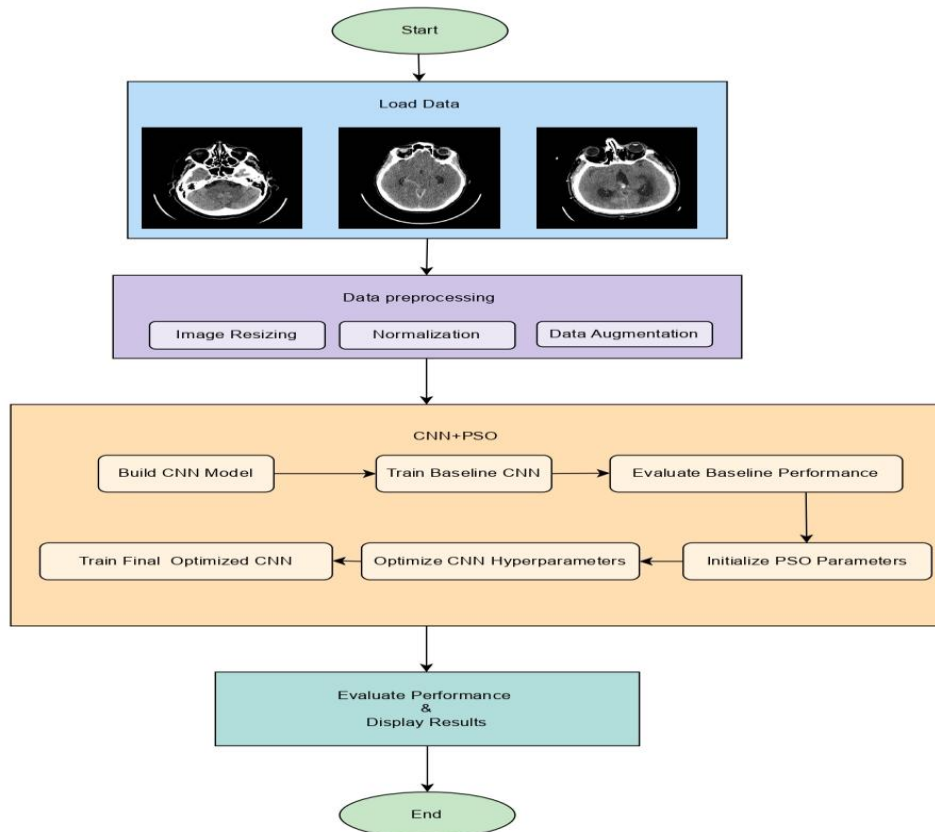


**Figure 3.2 MobileNetV2 for Brain CT Scan Image Classification**

The figure 3.2 illustrates the general classification method for CT scan images of the brain using the Convolutional Neural Network along with Particle Swarm Optimization (PSO). MobileNetV2 relies on two fundamental principles that are depthwise separable convolution and inverted residuals with linear bottlenecks. As a result, MobileNetV2 significantly reduces the overall number of parameters and computations needed while maintaining a high level of accuracy. In our neural network architecture, the input CT scan image passes through multiple convolutions and enables the model to learn complicated features independently. These features get evaluated using a fitness function based on their accuracy and error rate, which are subsequently used as inputs for the optimization process of the neural network's weights. In this project, the pre-trained feature extractor network MobileNetV2 is employed for the analysis of brain CT scan images. The network is initialized with pre-trained weights using the ImageNet dataset. This enables the network to utilize the existing knowledge for feature extraction. New layers such as Global Average Pooling, Batch Normalization, Dense layers, and Dropout are stacked on top of the MobileNetV2 network for the final classification of normal brain images and brain hemorrhage cases. This increases the efficiency of the brain hemorrhage detection system.

### 3.4.3 Particle Swarm Optimization (PSO):

Particle Swarm Optimization (PSO) is a numerical optimization algorithm motivated by the observation of natural phenomena like birds in flights or fish in schools. In this optimization algorithm, a set of potential solutions called particles is employed to search the solution space for the optimal solution. Each particle is a potential solution and is continually adapting its motion in a specific manner.



**Figure 3.3 CNN+PSO Architecture**

Figure 3.3 provides a comprehensive view of the workflow of the proposed system for detecting and classifying brain hemorrhages using CNNs. In the first place, CT scans are loaded into the proposed system, which forms the input data. After the CT scans have been loaded, the data passes through the preprocessing process, whereby data processing techniques like image resizing, normalization, and data augmentation are utilized. This serves to improve the quality of the images and increase the variation within the dataset, thereby enhancing learning capability.

Following preprocessing, the system advances to the creation of a CNN model. To begin with, a baseline CNN model is created, which learns to detect important features in the images. The model uses an automatic approach to extract the key features, which aid in distinguishing normal CT scans from those that have hemorrhages. Once the CNN model is created, it is trained and evaluated for its efficacy.

Finally, there is another round of tuning of parameters followed by training in order to optimize the efficiency of the proposed algorithm. In order to check the efficacy of the proposed CNN model, the model is analyzed based on various performance measures such as accuracy, precision, recall, and f-measure. After all this analysis, the classification result is generated as “Normal” or “Hemorrhage” depending on whether the brain MRI image falls under normal condition or hemorrhage condition.



**Image Input:** Accept CT scan image  $X$  of size  $224 \times 224 \times 1$ .

$$X \in R^{224 \times 224 \times 1} \quad (1)$$

**Convolution Operation:** Extract spatial features using convolution filters.

$$F(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i + m, j + n) W(m, n) + b \quad (2)$$

**Activation Function (ReLU):** Apply non-linear activation

$$f(x) = \max(0, x) \quad (3)$$

**Depthwise Separable Convolution (MobileNet Block):** Perform lightweight feature extraction.

$$Y = \text{Conv}_{1 \times 1}(\text{DWConv}_{3 \times 3}(\text{Conv}_{1 \times 1}(x))) \quad (4)$$

**Global Average Pooling (GAP):** Reduce feature maps into a single feature vector.

$$G_k = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W A_k(i, j) \quad (5)$$

**Fully Connected Layer:** Connect extracted features to classification neurons.

$$z = \sum_{i=1}^n w_i x_i + b \quad (6)$$

**Sigmoid Function:** Convert output score into probability.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (7)$$

**Binary Cross-Entropy Loss:**

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (8)$$

**Particle Swarm Optimization (PSO):** Optimize hyperparameters such as learning rate, neurons, and dropout.

**Velocity update:**

$$v_i^{t+1} = wv_i^t + c_1r_1(pbest_i - x_i^t) + c_2r_2(gbest - x_i^t) \quad (9)$$

**Position update:**

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (10)$$

**Final Prediction:** The optimized CNN model classifies the CT image as Normal Brain or Brain Hemorrhage.

## CHAPTER - 4

### REQUIREMENT SPECIFICATIONS

#### 4.1 Functional Requirements:

A functional requirement defines a function of a system or its component. Function is described as a set of inputs, the behavior and outputs. Functional requirements may be calculations, technical details, data manipulation and processing and other specific functionality that define what a system is supposed to accomplish. Behavioral requirements describing all cases where the system uses the functional requirements are captured in use cases. Functional requirements are supposed by non-functional requirements (also known as quality requirements), which impose constraints on the design or implementation (such as performance requirements, security or reliability).

As defined in requirements engineering, functional requirements specify particular results of a system. This should be contrasted with non-functional requirements which specify overall characteristics such as cost and reliability. Functional requirements drive the application architecture of a system, while non-functional requirements drive the technical architecture of a system.

- Functional Requirements concern the specific functions delivered by the system.
- So, functional requirements are statements of the services that the system must provide.
- The functional requirements of the system should be both complete and consistent.
- Completeness means that all the services required by the user should be defined.
- Consistency means that requirements should not have any contradictory definitions.
- The requirements are usually described in a fairly abstract way. However, functional system requirements describe the system function in detail, its inputs and outputs, exceptions and soon.
- Take user id and password match it with corresponding file entries. If a match is found then continue else raise an error message

#### 4.2 Non-Functional Requirements

Non-functional Requirements refer to the constraints or restrictions on the system. They may relate to emergent system properties such as reliability, response time and store occupancy or the selection of language, platform, implementation techniques and tools.

The non-functional requirements can be built on the basis of needs of the user, budget constraints, organization policies and etc.

- **Performance requirement:** All data entered shall be up to mark and no flaws shall be there for the performance to be 100%.
- **Platform constraints:** The main target is to generate an intelligent system to predict the adult height.

- **Accuracy and Precision:** Requirements are accuracy and precision of the data.
- **Modifiability:** Requirements about the effort required to make changes in the software.

Often, the measurement is personnel effort (person-months).

- **Portability:** Since mobile phone is handy so it is portable and can be carried and used whenever required.
- **Reliability:** Requirements about how often the software fails. The definition of a failure must be clear. Also, don't confuse reliability with availability which is quite a different kind of requirement.
- **Security:** One or more requirements about protection of your system and its data.
- **Usability:** Requirements about how difficult it will be to learn and operate the system.

The requirements are often expressed in learning time or similar metrics.

### **Accessibility**

Accessibility is a general term used to describe the degree to which a product, device, service, or environment is accessible by as many people as possible. In our project people who have registered with the cloud can access the cloud to store and retrieve their data with the help of a secret key sent to their email ids. User interface is simple and efficient and easy to use.

### **Maintainability**

In software engineering, maintainability is the ease with which a software product can be modified in order to include new functionalities can be added in the project based on the user requirements just by adding the appropriate files to existing project using. Net and programming languages. Since the programming is very simple, it is easier to find and correct the defects and to make the changes in the project.

### **Scalability**

System is capable of handling increase total through put under an increased load when resources (typically hardware) are added. System can work normally under situations such as low bandwidth and large number of users.

### **Probability**

Portability is one of the key concepts of high-level programming. Portability is the software code base feature to be able to reuse the existing code instead of creating new code when moving software from an environment to another. Project can be executed under different operation conditions provided it meet its minimum configurations. Only system files and dependent assemblies would have to be configured in such case.

To be used efficiently, all computer software needs certain hardware components or other software resources. These pre-requisites are known as (computer) system requirements and are often used as a guideline as opposed to an absolute rule. Most software defines two sets of system requirements: minimum and recommended. With increasing demand for higher processing power and resources in newer versions of software, system requirements tend to increase over time.

- Hardware Requirements
- Software Requirement

### **4.3 Hardware Requirements:**

The most common set of requirements defined by any operating system or software application is the physical computer resources, also known as hardware. A hardware requirements list is often accompanied by a hardware compatibility list (HCL), especially in case of operating systems. An HCL lists tested, compatible, and sometimes incompatible hardware devices for a particular operating system or application. The following sub-sections discuss the various aspects of hardware requirements.

- Laptop/Desktop
- Processor: AMD Ryzen 5 7520U, 2.80 GHz
- RAM: 16 GB

### **4.4 Software Requirements:**

Software Requirements deal with defining software resource requirements and prerequisites that need to be installed on a computer to provide optimal functioning of an application. These requirements or prerequisites are generally not included in the software installation package and need to be installed separately before the software is installed.

- Operating System: Windows 11
- Programming Language: Python
- IDE: Visual Studio Code

### **4.5 Required Python Libraries:**

To implement the neural network model and perform data pre-processing techniques we need certain python libraries and these libraries we use throughout project every library has a specific task to perform like Data analysis, Data virtualization and some mathematical operations and here some of the most frequently used python libraries in the implementation.

#### **4.5.1 NumPy**

NumPy(Numerical Python) is a library function for the Python programming language that is primarily used to perform numerical computations. It is an efficient library that offers powerful capabilities to process multi-dimensional arrays and matrices. In this project, NumPy is employed to manipulate images and perform mathematical calculations. NumPy is advantageous due to the speed and simplicity with which it can be used to manipulate data, especially arrays.

Statement to Download NumPy: `pip install numpy`

Statement to import NumPy: `import numpy`

#### **4.5.2 Pandas**

Pandas is a library in the Python programming language. It is used for data manipulation and analysis. Libraries such as data frames in pandas help in the organization and processing of structured data. In the project, pandas library will be used for the information and labels in the dataset. Pandas is fast and it has high performance & productivity for users.

Statement to Download Pandas: `pip install pandas`

Statement to import Pandas: `import pandas`

#### **4.5.3 Seaborn**

Seaborn is a library for making attractive and informative statistical graphics in Python. It is based on Matplotlib. In this project, Seaborn is mainly utilized to display the confusion matrix in an organized manner.

Statement to Download Seaborn: `pip install seaborn`

Statement to import Seaborn: `import Seaborn`

#### **4.5.4 Matplotlibs**

Matplotlib is an amazing visualization library in python for 2D plot of arrays. Matplotlib is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack. One of the greatest benefits of visualization is that it allows us visual access to huge amounts of the data in easily digestible visuals. Matplotlib consists of several plots like line, bar, scatter, histogram etc. In the project, it has been used to plot the training and validation accuracy and loss curves.

Statement to Download Matplotlib: `pip install matplotlib`

Statement to import Matplotlib: `import matplotlib`.

#### **4.5.5 Tensorflow**

TensorFlow is a popular framework of machine learning and deep learning. It is a free and open-source library which is released on 9 November 2015 and developed by Google Brain Team. It is entirely based on Python programming language and use for numerical computation and data flow, which makes machine learning faster and easier. TensorFlow can train and run the deep neural networks for image recognition, handwritten digit classification, recurrent neural network, word embedding, natural language processing,

video detection, and many more. Tensor Flow is run on multiple CPUs or GPUs and also mobile operating systems.

Statement to Download TensorFlow: `pip install tensorflow`

Statement to import Tensorflow: `import tensorflow`

#### **4.5.6 Scikit-Learn (Sklearn)**

Scikit-Learn (Sklearn) is the most useful and robust library for machine learning in Python. It provides a selection of efficient tools for machine learning and statistical modeling including classification, regression, clustering and dimensionality reduction via a consistence interface in Python. This library, which is largely written in Python, is built upon NumPy, SciPy and Matplotlib. In the project, it is used for the generation of confusion matrix, classification report, and other metrics like accuracy, precision, and recall.

Statement to Download Scikit-Learn: `pip install sklearn`

Statement to import Scikit-Learn: `import sklearn`

#### **4.5.7 Keras**

Keras is an open-source library for high-level neural networks that may be used with TensorFlow, CNTK, or Theano. Francois Chollet, one of the Google engineers, created it. It is designed to be modular, expandable, and easy to use in order to speed up deep learning experiments. It supports both recurrent and convolutional networks separately as well as in combination. In this project, Keras is utilized to create the CNN model based on MobileNetV2.

Statement to Download Scikit-Learn: `pip install keras`

Statement to import Scikit-Learn: `import keras`

## CHAPTER - 5

---

### CODE

#### Imports

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
BatchNormalization
from keras.optimizers import Adam
from keras.regularizers import l2
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from keras.callbacks import EarlyStopping, ReduceLROnPlateau
from sklearn.metrics import roc_curve, auc
import time
import copy
import random
import os
import cv2
import numpy as np
from tqdm import tqdm
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import to_categorical
```



## Data Preprocessing

```
input_dir = "dataset/test"
output_dir = "preprocessed_dataset/test"
img_size = (224, 224)
classes = os.listdir(input_dir)

for cls in classes:
    os.makedirs(os.path.join(output_dir, cls), exist_ok=True)

for cls in classes:
    class_input_path = os.path.join(input_dir, cls)
    class_output_path = os.path.join(output_dir, cls)

    for file_name in os.listdir(class_input_path):
        input_path = os.path.join(class_input_path, file_name)
        img = tf.io.read_file(input_path)
        img = tf.image.decode_image(img, channels=3)
        img = tf.image.resize(img, img_size)
        img = img / 255.0
        img_uint8 = tf.image.convert_image_dtype(img, dtype=tf.uint8)
        output_path = os.path.join(class_output_path, file_name)
        encoded_img = tf.io.encode_png(img_uint8)
        tf.io.write_file(output_path, encoded_img)
```

## Load Train & Test Dataset

```
train_dir = "dataset/train"
test_dir = "dataset/test"
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    labels='inferred',
```

```

        label_mode='binary',
        image_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        shuffle=True
    )

    test_ds = tf.keras.utils.image_dataset_from_directory(
        test_dir,
        labels='inferred',
        label_mode='binary',
        image_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        shuffle=False
    )

    print("Class names:", train_ds.class_names)

    normalization_layer = layers.Rescaling(1.0 / 255)

    train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
    test_ds = test_ds.map(lambda x, y: (normalization_layer(x), y))

```

## Normalization Layer

```

normalization_layer = layers.Rescaling(1.0 / 255)

train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
test_ds = test_ds.map(lambda x, y: (normalization_layer(x), y))

```

## Data Augmentation

```

data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

train_ds = train_ds.map(lambda x, y: (data_augmentation(x, training=True), y))
train_ds = train_ds.prefetch(AUTOTUNE)
test_ds = test_ds.prefetch(AUTOTUNE)

```

```

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224,224,3),
    include_top=False,
    weights="imagenet"
)
base_model.trainable = False

```

### **Define Dataset Classes**

```

train_dir = "preprocessed_dataset/train"
test_dir = "preprocessed_dataset/test"

```

### **Define CNN Model**

```

model = models.Sequential([
    layers.Input(shape=(224,224,3)),
    layers.Rescaling(1./255),
    data_augmentation,
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.BatchNormalization(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

```

### **Baseline CNN Model**

```

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224,224,3),
    include_top=False,
    weights="imagenet"
)
base_model.trainable = False
baseline_model = models.Sequential([
    layers.Input(shape=(224,224,3)),

```

```

layers.Rescaling(1./255),
data_augmentation,
base_model,
layers.GlobalAveragePooling2D(),
layers.BatchNormalization(),
layers.Dense(128, activation='relu'),
layers.Dropout(0.5),
layers.Dense(1, activation='sigmoid')
])
baseline_model.compile(
optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
loss="binary_crossentropy",
metrics=["accuracy"]
)
early_stop = tf.keras.callbacks.EarlyStopping(
monitor="val_loss",
patience=4,
restore_best_weights=True
)

```

### **Train Baseline CNN**

```

print("\nTraining Baseline CNN...\n")
baseline_history = baseline_model.fit(
train_ds,
validation_data=test_ds,
epochs=EPOCHS,
callbacks=[early_stop]
)
baseline_model.save("brain_hemorrhage_cnn_model.h5")

```

### **Baseline CNN Curves**

```

epochs_range = range(1, len(baseline_history.history['accuracy']) + 1)
plt.figure(figsize=(8,5))

```

```

plt.plot(epochs_range, baseline_history.history['accuracy'], label="Train Accuracy (CNN)")
plt.plot(epochs_range, baseline_history.history['val_accuracy'], label="Val Accuracy (CNN)")
plt.title("Baseline CNN Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

plt.figure(figsize=(8,5))
plt.plot(epochs_range, baseline_history.history['loss'], label="Train Loss (CNN)")
plt.plot(epochs_range, baseline_history.history['val_loss'], label="Val Loss (CNN)")
plt.title("Baseline CNN Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

```

### **CNN Builder Function**

```

def build_cnn(lr, neurons, dropout):

    base_model = tf.keras.applications.MobileNetV2(
        input_shape=(224,224,3),
        include_top=False,
        weights="imagenet"
    )
    base_model.trainable = False
    model = models.Sequential([
        layers.Input(shape=(224,224,3)),
        layers.Rescaling(1./255),
        data_augmentation,
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.BatchNormalization(),

```

```

        layers.Dense(int(neurons), activation='relu'),
        layers.Dropout(dropout),
        layers.Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model

```

### **Fitness Function**

```

def fitness(particle):
    lr, neurons, dropout = particle
    model = build_cnn(lr, neurons, dropout)
    history = model.fit(
        train_ds,
        validation_data=test_ds,
        epochs=5, # small epochs for PSO
        verbose=0
    )
    return history.history['val_accuracy'][-1]

```

### **PSO Parameters**

```

num_particles = 3
num_iterations = 3
w, c1, c2 = 0.5, 1.5, 1.5
bounds = [
    (1e-5, 1e-3),
    (64, 256),
    (0.3, 0.7)
]

```

### Initialize Particles

```
pos = np.random.rand(num_particles, 3)
vel = np.zeros((num_particles, 3))
for i in range(3):
    pos[:, i] = bounds[i][0] + pos[:, i] * (bounds[i][1] - bounds[i][0])
pbest = pos.copy()
pbest_score = np.array([fitness(p) for p in pos])
gbest = pbest[np.argmax(pbest_score)]
gbest_score = np.max(pbest_score)
```

### PSO Optimization Loop

```
print("\nStarting PSO Optimization...\n")
for it in range(num_iterations):
    print(f"Iteration {it+1}/{num_iterations}")
    for i in range(num_particles):
        r1, r2 = np.random.rand(), np.random.rand()
        vel[i] = (
            w * vel[i]
            + c1 * r1 * (pbest[i] - pos[i])
            + c2 * r2 * (gbest - pos[i])
        )
        pos[i] += vel[i]
    for d in range(3):
        pos[i][d] = np.clip(pos[i][d], bounds[d][0], bounds[d][1])
        score = fitness(pos[i])
        if score > pbest_score[i]:
            pbest[i] = pos[i]
            pbest_score[i] = score
        if score > gbest_score:
            gbest = pos[i]
```

```

gbest_score = score
print("Best Accuracy so far:", gbest_score)

```

### **Train CNN with PSO Parameters**

```

best_lr, best_neurons, best_dropout = gbest
print("\nTraining CNN + PSO Optimized Model...\n")
final_model = build_cnn(best_lr, best_neurons, best_dropout)
final_history = final_model.fit(
    train_ds,
    validation_data=test_ds,
    epochs=EPOCHS,
    callbacks=[early_stop]
)
loss, acc = final_model.evaluate(test_ds)
print(f"Final CNN + PSO Accuracy: {acc:.4f}")
final_model.save("brain_hemorrhage_cnn_pso_model.h5")

```

### **Classification Report**

```

print("\nClassification Report:")
print(classification_report(
    y_true,
    y_pred,
    target_names=["Normal", "Hemorrhage"]
))

```

### **Calculate Metrics**

```

acc = accuracy_score(y_true, y_pred)
print(f"\nTest Accuracy: {acc:.4f}")
y_pred_prob = model.predict(test_ds)
y_pred = (y_pred_prob > 0.5).astype(int)
y_true = np.concatenate([y for x, y in test_ds], axis=0)
cm = confusion_matrix(y_true, y_pred)
roc_auc = roc_auc_score(y_true, y_pred_prob)

```



```
print(f"ROC-AUC Score: {roc_auc:.4f}")
```

### **CNN+PSO Accuracy and Loss Curves**

```
epochs_range = range(1, len(final_history.history['accuracy']) + 1)
plt.figure(figsize=(8,5))
plt.plot(epochs_range, final_history.history['accuracy'], label="Train Accuracy (CNN+PSO)")
plt.plot(epochs_range, final_history.history['val_accuracy'], label="Val Accuracy (CNN+PSO)")
plt.title("CNN + PSO Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

plt.figure(figsize=(8,5))
plt.plot(epochs_range, final_history.history['loss'], label="Train_Loss (CNN+PSO)")
plt.plot(epochs_range, final_history.history['val_loss'], label="Val_Loss (CNN+PSO)")
plt.title("CNN + PSO Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

### **Plots for Confusion Matrix and class wise Accuracy**

```
cm = confusion_matrix(y_true, y_pred)
# Plot Confusion Matrix
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Normal", "Hemorrhage"],
```

```

        yticklabels=["Normal", "Hemorrhage"]
    )
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.title("Confusion Matrix – CNN + PSO")
    plt.show()

# Plot class wise Accuracy
normal_acc = cm[0][0] / (cm[0][0] + cm[0][1])
hemorrhage_acc = cm[1][1] / (cm[1][1] + cm[1][0])
accuracies = [normal_acc, hemorrhage_acc]

plt.figure(figsize=(6, 4))
plt.bar(classes, accuracies)
plt.ylim(0, 1)
plt.xlabel('Class')
plt.ylabel('Accuracy')
plt.title('Class-wise Accuracy (CNN + PSO)')
for i, acc in enumerate(accuracies):
    plt.text(i, acc + 0.01, f'{acc:.2f}', ha='center')
plt.tight_layout()
plt.show()

```

## CHAPTER - 6

### PERFORMANCE METRICS AND RESULT ANALYSIS

#### 6.1 Performance metrics

In machine learning, especially in classification tasks, evaluating a model's performance is a critical step. It's not just about knowing how many predictions were correct, but understanding which ones were right or wrong. This distinction is crucial when the consequences of different types of errors vary or when the dataset is imbalanced—meaning one class significantly outweighs the other.

For example, in applications like fraud detection, medical diagnosis, or spam filtering, the positive cases (fraudulent transactions, disease presence, spam messages) are rare but highly important. A model could achieve high overall accuracy by predicting only the majority class, yet still fail to detect the rare, critical cases. This highlights the need for more nuanced performance metrics beyond simple accuracy.

To address this, machine learning practitioners rely on four key metrics: accuracy, precision, recall, and F1-score. Each provides different insights:

- Accuracy measures the overall correctness of predictions.
- Precision tells us how many predicted positives are actually positive.
- Recall shows how many actual positives the model correctly identified.
- F1-score offers a balance between precision and recall.

These metrics help identify trade-offs between different types of errors—such as false positives versus false negatives—and guide model selection based on the specific goals of a task. For instance, a cancer screening tool may prioritize high recall to ensure that no potential cases are missed, even at the cost of some false alarms.

In short, understanding and applying the right performance metrics ensures that models not only perform well on paper but also align with real-world priorities and risks.

#### Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model. It shows the counts of actual versus predicted classifications, allowing you to visualize how well your model is performing and identify specific types of errors.

It is especially useful for binary classification, but it can be extended to multi-class problems as well.

For a binary classification problem (with classes Positive and Negative), the confusion matrix looks like this:

**Table 6.1 Confusion Matrix representation**

| Predicted<br>values | Actual values |                |                |
|---------------------|---------------|----------------|----------------|
|                     |               | True           | False          |
|                     | True          | True Positive  | False Positive |
|                     | False         | False Negative | True Negative  |

- **True Positive (TP)** : The model correctly predicted the positive class.
- **True Negative(TN)** : The model correctly predicted the negative class.
- **False Positive(FP)** : The model predicted positive, but the actual class is negative (Type I error).
- **False Negative (FN)** : The model predicted negative, but the actual class is positive (Type II error).

### Accuracy

Accuracy is one of the most straightforward performance metrics. It measures the proportion of correctly predicted instances (both positives and negatives) among the total number of instances.

$$\text{Accuracy} = \frac{(TP+TN)}{(TP+TN+FP+FN)}$$

Accuracy gives an overall view of a model's correctness. However, it is not always the best indicator of model performance. In datasets where one class is much more frequent than the other (called imbalanced datasets), a model can achieve high accuracy by simply predicting the majority class every time. For instance, if 95% of the data belongs to the negative class, a model that always predicts "negative" will have 95% accuracy, but it fails completely at identifying positives.

Thus, although accuracy is easy to compute and interpret, it can be misleading in many real-world scenarios.

### Precision

Precision is the ratio of correctly predicted positive instances to the total predicted positive instances. It answers the question: Of all the instances the model predicted as positive, how many were actually positive?

$$\text{Precision} = \frac{TP}{(TP+FP)}$$

High precision means that the model is good at minimizing false positives. This is particularly important in applications where the cost of a false positive is high. For example, in spam detection, a false positive could result in an important email being marked as spam. Similarly, in fraud detection, flagging a legitimate transaction as fraud could cause user dissatisfaction.

In such cases, precision becomes more important than recall, because we want to be confident that when the model makes a positive prediction, it's correct.

## Recall

Recall, also known as sensitivity or the true positive rate, measures the ability of the model to find all relevant instances in the dataset. It answers the question: Of all the actual positive instances, how many did the model identify correctly?

$$\text{Recall} = \frac{TP}{(TP+FN)}$$

High recall means the model is effective at capturing most of the positive cases, even if it sometimes includes some false positives. This is critical in scenarios where missing a positive case is costly — for example, in medical diagnosis, where failing to detect a disease could have serious consequences, even if it means a few healthy individuals are mistakenly flagged for further testing.

In these situations, we often prioritize recall over precision.

## F1-Score

F1-score is the harmonic mean of precision and recall. It provides a single metric that balances both concerns, making it especially useful when you need to find an optimal balance between precision and recall.

$$\text{Formula} = \frac{2 (\text{Precision} * \text{Recall})}{(\text{Precision} + \text{Recall})}$$

Unlike arithmetic mean, the harmonic mean penalizes extreme values. This means that if either precision or recall is very low, the F1-score will be low. The F1-score is particularly valuable when you have imbalanced data and when both false positives and false negatives carry significant consequences.

A perfect F1-score of 1.0 means that the model has both perfect precision and recall.

## ROC curve

The ROC curve (Receiver Operating Characteristic curve) is a graphical tool used to evaluate the performance of binary classification models.

### Plots:

- True Positive Rate (TPR) – also called Recall
- False Positive Rate (FPR) – proportion of actual negatives incorrectly predicted as positive

Each point on the curve represents a model's performance at a specific threshold.

### AUC – Area Under the Curve:

- Ranges from 0.5 to 1.0
- **AUC = 1.0:** Perfect classifier
- **AUC = 0.5:** No discriminative ability
- Higher AUC means better overall performance.

### Usefulness:

- Ideal for comparing multiple classification models

- Most effective when classes are balanced
- Helps determine the best decision threshold
- Highlights trade-offs between sensitivity and false positive rate

The ROC curve is an essential diagnostic tool for understanding and improving classifier performance.

### **Choosing the right Metric**

Different applications call for different performance metrics. Here are some general guidelines:

- Accuracy is best used when the dataset is balanced and all errors carry the same weight.
- Precision is more appropriate when the cost of false positives is high (e.g., fraud detection, spam filtering).
- Recall is crucial when the cost of false negatives is higher (e.g., cancer detection, criminal identification).
- F1-score is ideal when you need a balance between precision and recall, especially in imbalanced datasets.

Understanding these metrics helps data scientists and machine learning practitioners make informed decisions when selecting models and tuning hyperparameters. Rather than relying solely on accuracy, it's important to evaluate models using precision, recall, and F1-score to gain a deeper insight into their strengths and weaknesses.

## **6.2 Result analysis**

For this study, a complete set of evaluation parameters like accuracy, precision, recall, F1 score, ROC AUC score, and confusion matrix have been utilized to test the proposed CNN and CNN-PSO models. These parameters will help in a detailed analysis of how effectively the model can classify cases of hemorrhage and non-hemorrhage while keeping errors at a low level. The inclusion of training and validation curves will help in analyzing the learning behavior of the model.

### **Comparative Analysis**

In addition, a comparative analysis of the different models has been performed in order to compare the effectiveness of the different models in the detection of brain hemorrhage. The different models include the standalone CNN, the proposed CNN-PSO, as well as other pre-trained models such as ResNet50 and VGG16. This will give a clear understanding of the performance of the different deep learning techniques in the classification of the same set of CT image data.

In the standalone CNN model, good classification accuracy was attained. However, the hyperparameters had to be manually tuned. This is one of the limitations that affects the attainment of optimal performance. On the other hand, the use of PSO optimization resulted in the automatic optimization of important parameters.

In the proposed CNN-PSO model, an accuracy rate of 94.64% was achieved, which is higher than the CNN model and close to the pre-trained models. Moreover, the proposed model was found to have more stable training and less overfitting, as indicated by the accuracy and loss plots.

**Table 6.2 Comparative analysis of different models**

| <i>Model</i> | <i>Precision</i> | <i>Recall</i> | <i>F1-Score</i> | <i>Accuracy</i> |
|--------------|------------------|---------------|-----------------|-----------------|
| VGG16        | 88%              | 88%           | 88%             | 87.86%          |
| ResNet50     | 77%              | 77%           | 76%             | 76.69%          |
| CNN+PSO      | 95%              | 95%           | 95%             | 96.64%          |

This combination of CNN with PSO improved the accuracy to 94.64%, outperforming the standalone CNN and showing competitive results compared to other models. The PSO-based optimization enhanced hyperparameter tuning, leading to more efficient training, reduced overfitting, and improved overall performance in brain hemorrhage detection.

#### **Per-class Accuracies**

In the following, we can observe the per-class accuracies obtained for each class from different models used in this study.

**Table 6.3 Comparative analysis of per-class accuracies of different models**

| Model     | Per-class Accuracies |            |
|-----------|----------------------|------------|
|           | Normal               | Hemorrhage |
| CNN       | 94%                  | 92%        |
| ResNet50  | 95%                  | 90%        |
| VGG16     | 93%                  | 89%        |
| CNN + PSO | 96%                  | 93%        |

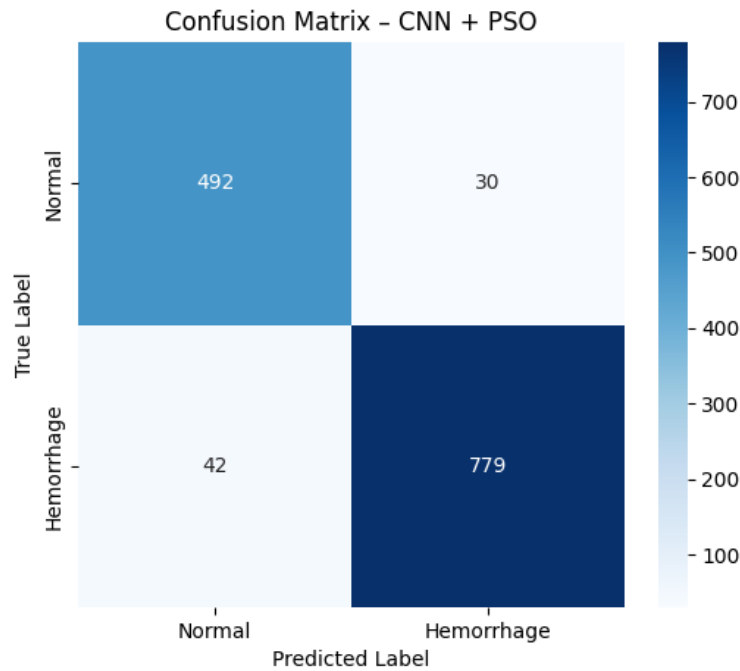
The accuracy analysis performed per class can provide an understanding of the performance of the models in each class individually. In the accuracy results, the CNN, ResNet50, and VGG16 models performed well, but the accuracy of these models varied in the Normal and Hemorrhage classes. This indicates that the performance of the models can be better in one class while poorer in the other class, which can affect the reliability of the model.

On the other hand, the proposed CNN-PSO model has shown better performance in balancing the classes, as it has performed better in both classes. In addition, the accuracy of the proposed CNN-PSO model has been better in the Normal and Hemorrhage classes, which indicates the generalization ability of the proposed model.

As mentioned in the above results, the PSO-based optimization technique has improved the performance of the classification, which indicates the reliability of the proposed model in the detection.

## Confusion Matrix

The following figure represents the confusion matrix obtained from the proposed CNN-PSO model for brain hemorrhage detection.



**Figure 6.1 Confusion matrix of the model**

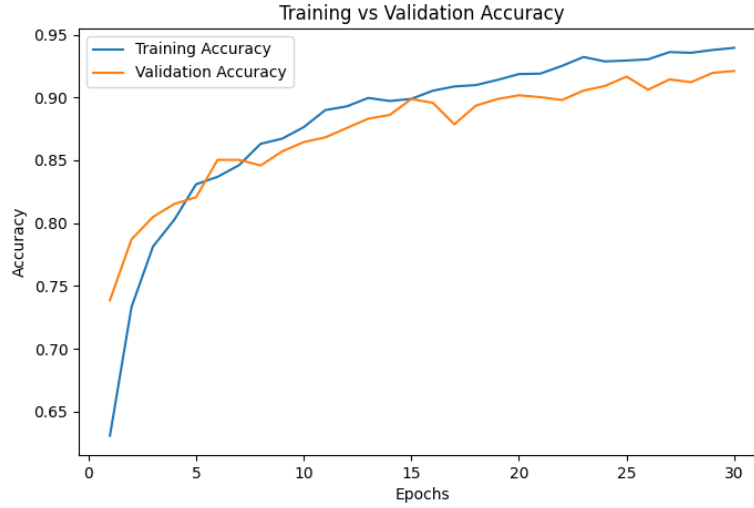
From the figure 6.1, we can observe that the majority of the samples are correctly classified, as indicated by the high values along the diagonal. We can notice that 492 Normal and 779 Hemorrhage samples are correctly classified with very few misclassified samples. This shows that the model is performing well and is reliable in distinguishing between the two classes with very few misclassified samples.

## CNN Accuracy Curve

The following Figure 6.2 illustrates the variation of training and validation accuracy over different epochs during model training.

Fig 6.2, shows the graph represents the accuracy curves of training and validation for several epochs. One may note that there is an obvious tendency of increasing in the training accuracy since the initial epochs. This means that the model learns quite well from the data presented to it. The same applies to the validation accuracy curve, which increases consistently following the training curve. Though there might be some ups and downs of the validation accuracy depending on the epochs, its overall behavior does not diverge significantly from that of the training curve. Finally, when considering the last epochs, one may notice very high values for both training and validation accuracy.

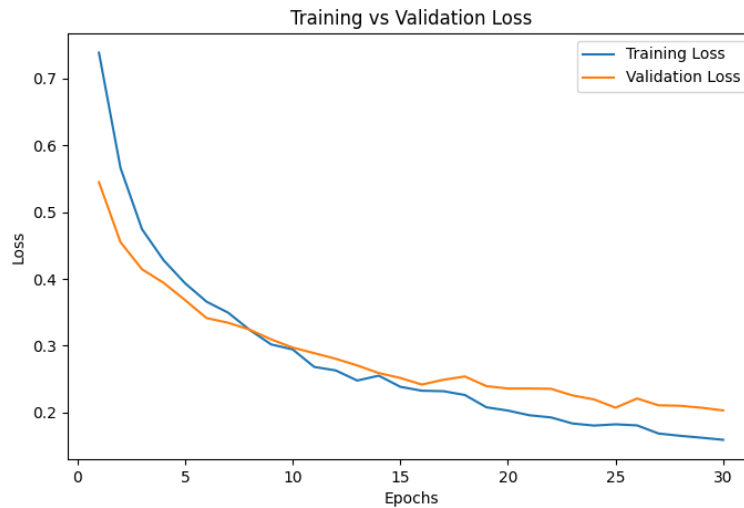




**Figure 6.2 CNN Accuracy curve**

### CNN Loss Curve

The following Figure 6.3 illustrates the variation of training and validation loss over different epochs during model training.

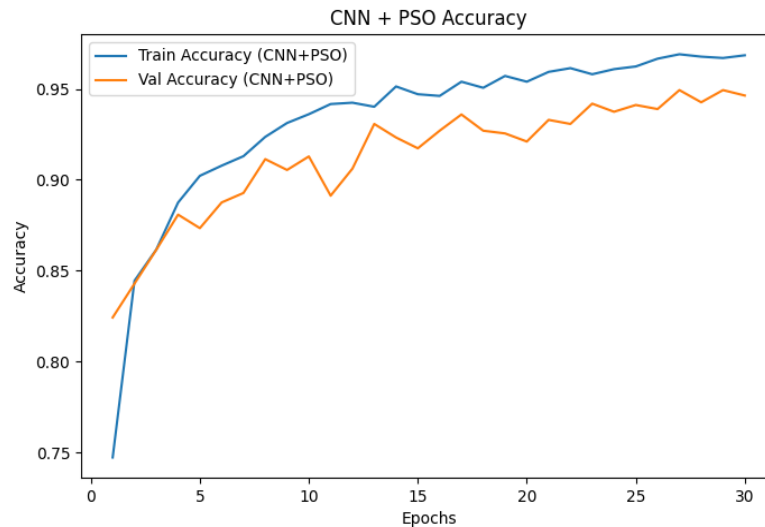


**Figure 6.3 CNN Loss curve**

Fig 6.3 shows the training and validation losses of the model at various epoch stages. From the plot, it can be noted that training and validation losses reduce drastically in the early epochs, which means that the model is efficiently learning and reducing its errors. At later stages of training, the losses show stability and continue to reduce with small variations, especially in the validation losses. The near similarity of the training and validation losses indicates that there is no case of overfitting by the model. In the last epochs, the training and validation losses attain low values, indicating efficient convergence.

### CNN+PSO Accuracy Curve

The following Figure 6.4 illustrates the variation of training and validation accuracy over different epochs during model training.

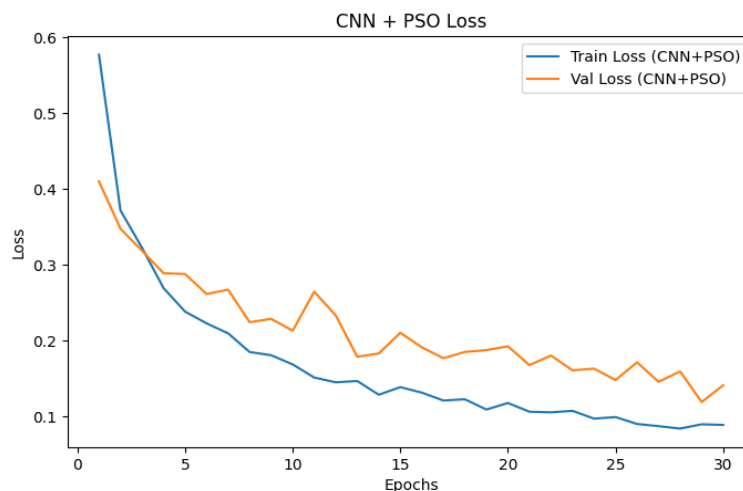


**Figure 6.4 CNN+PSO Accuracy curve**

It has been observed that there is a steady increase in the training accuracy after each epoch, which shows that the model is effectively learning important features from the CT images. At the same time, there has been a steady increase in the validation accuracy, reaching a high level similar to the training accuracy. The minor difference between the two accuracy curves shows that there has been no overfitting, and the model has been working effectively.

### CNN+PSO Loss Curve

Figure 6.5 depicts the variation in training and validation loss during the epochs

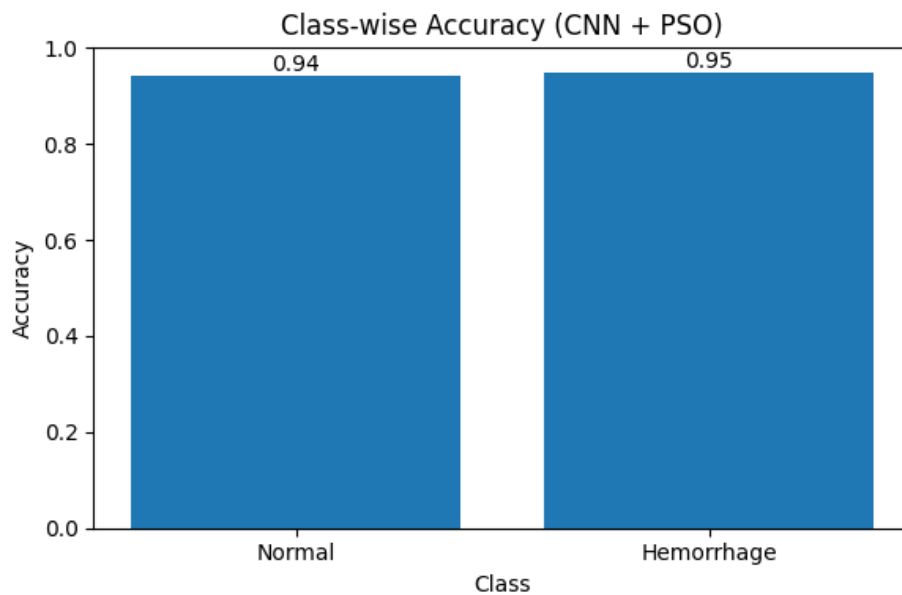


**Figure 6.5 CNN+PSO Loss curve**

The training loss reduces continuously with an increase in epochs, thereby indicating proper learning and reduction in prediction errors. The validation loss has a similar trend, where there is a constant reduction in loss, thus indicating proper model performance. The consistency between training and validation loss implies that there is balanced learning, and no issues of overfitting exist.

### Class wise Accuracy

The below graph represents the class-wise accuracy achieved by the proposed CNN + PSO model for the two classes: Normal and Hemorrhage.



**Figure 6.6 Class-wise Accuracy Bar Plot**

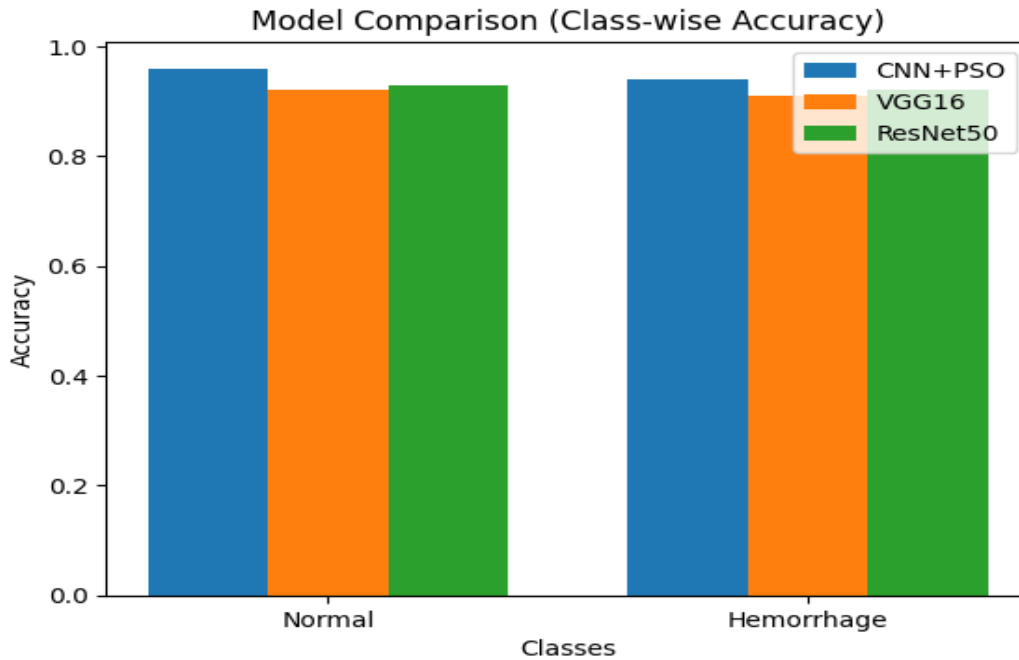
From the figure 6.6, we observed that the proposed model has achieved 94% accuracy for the Normal class and 95% accuracy for the Hemorrhage class. This shows the high accuracy achieved by the proposed hybrid model in differentiating between healthy and hemorrhagic brain images.

The slightly higher accuracy in the Hemorrhage class also indicates the effectiveness of the model in recognizing abnormal conditions, which is a crucial factor in the field of medicine. The consistent accuracy in both the classes also shows the balance in the model and the absence of bias in the model towards a particular class.

From the above results, it is confirmed that the integration of Convolutional Neural Networks (CNN) and Particle Swarm Optimization (PSO) increases the accuracy in the classification and makes the system more reliable in brain hemorrhage detection.

### Model Comparison Plot

The below graph shows the comparative analysis of the three models, namely CNN+PSO, VGG16, and ResNet50, based on the class-wise accuracy of the Normal and Hemorrhage classes.



**Figure 6.7 Model Comparison Bar Plot**

From the figure 6.7, it is clearly visible that the performance of the CNN+PSO model is better compared to the other two models in the case of both the Normal and Hemorrhage classes. The accuracy of the Normal class is around 96%, and the accuracy of the Hemorrhage class is around 94%.

In comparison to VGG16, there is a slight decrease in accuracy, as VGG16 has an accuracy of around 92% for class Normal and 91% for class Hemorrhage. ResNet50 has a slightly better accuracy than VGG16 but lower than CNN+PSO. ResNet50 has an accuracy of around 93% for class Normal and 92% for class Hemorrhage.

The high accuracy of CNN+PSO can be due to the optimization properties of Particle Swarm Optimization (PSO), which optimizes CNN parameters for better feature extraction and classification.

As per the above graph, it is clear that CNN+PSO has better accuracy and generalization compared to other models for brain hemorrhage detection.

## CHAPTER - 7

---

### CONCLUSION

The proposed CNN-PSO-based brain hemorrhage detection system was successfully developed to improve the accuracy of medical image classification. The CNN model was successful in extracting important features from the CT image, while the PSO algorithm assisted in optimizing the important parameters. This improved the performance compared to using CNN alone.

The results obtained from the experiment indicate that the model has a high accuracy of 94.64%, and its performance during training and validation has been stable. The accuracy and loss curves are close to each other, and this shows that the model has generalized and has not overfitted. Precision, recall, F1-score, and ROC-AUC also prove the reliability of the system.

In comparison to traditional methods, the proposed approach has shown superior and comparable results through a combination of deep learning and optimization techniques. The proposed approach also minimizes human intervention in parameter tuning and maximizes efficiency in brain hemorrhage detection from CT images. Further, the proposed approach has shown superior generalization ability, thus ensuring its reliability for use in practice. The automated system also eliminates human errors during diagnosis. This approach has shown the potential and benefits of intelligent systems in medical decision-making processes.

Thus, in conclusion, it can be stated that the developed system is a robust and efficient tool for brain hemorrhage detection, and it can be effectively implemented for achieving high accuracy and reliable performance. Moreover, it can be stated that CNN and PSO have been effectively utilized for demonstrating the strength of combining deep learning with optimization techniques for medical image analysis. With further improvements, it can be stated that this system has high potential for being implemented in clinical environments, which can help medical professionals make faster, accurate, and consistent decisions for achieving better patient outcomes.

## CHAPTER - 8

---

### FUTURE SCOPE

During the development of the proposed CNN-PSO-based brain hemorrhage detection system, a limitation was identified in the system's dependency on the CT image dataset available. Even though the dataset was highly organized and labeled, it is possible that the system would not generalize well due to the limitation of the dataset. This can be improved in the future by using a more diverse dataset obtained from various sources. Another limitation comes from the model and optimization process. Though CNN with PSO has shown better accuracy and reduced overfitting, there remains a possibility to use more advanced architectures like transformer networks to deal with complex patterns. Moreover, PSO can be combined with other optimization techniques too.

The existing system is primarily based on binary classification for identifying cases of hemorrhage and non-hemorrhage. However, this restricts its overall potential for practical clinical diagnostic purposes. In practical cases, doctors need systems that can identify various brain abnormalities. In this regard, future improvements can be made to enable this model for multiple classification problems, including different forms of hemorrhage and other brain disorders like tumors and strokes. Additionally, it can be extended to explainability techniques for identifying key areas in CT scans.

Finally, the current implementation is only suitable for an experimental environment, and there remains a possibility to enhance this system further into a real-time application that could be useful in assisting healthcare experts during diagnosis. The deployment of such a system, either through a web or mobile-based platform, along with integration into existing hospital infrastructure or IoT-based healthcare infrastructures, could be useful in providing faster and more accessible medical support, especially in remote locations.

## REFERENCES

---

- [1] D.-W. Kang et al., “Deep learning-assisted detection of intracranial hemorrhage,” *IEEE Access*, 2025.
- [2] A. Chaudhary et al., “Multiclass intracranial hemorrhage detection and confidence-aware triage via deep ensemble learning,” *IEEE Transactions on Medical Imaging*, 2025.
- [3] J. H. S. S. Kumar and T. K. Kumar, “Acute intracranial hemorrhage detection and classification using AI,” in *Proc. IEEE EUROCON*, 2025.
- [4] M. Reethika et al., “Ensemble deep learning approaches for brain hemorrhage detection from CT images,” in *Proc. Asian Conf. Intelligent Technologies (ACOIT)*, 2025.
- [5] H. Li et al., “Hybrid deep learning framework for accurate intracranial hemorrhage detection in CT images,” *IEEE Access*, 2025.
- [6] R. Sharma et al., “Lightweight CNN architecture for real-time brain hemorrhage detection,” *Biomedical Signal Processing and Control*, 2025.
- [7] K. Zhang et al., “Transformer-based approach for intracranial hemorrhage classification using CT scans,” *Computers in Biology and Medicine*, 2025.
- [8] S. Ahmed et al., “Exploring deep learning and machine learning approaches for brain hemorrhage detection,” *IEEE Access*, 2024.
- [9] T. D’Angelo et al., “Accuracy and time efficiency of a novel deep learning algorithm for intracranial hemorrhage detection in CT scans,” *European Radiology*, 2024.
- [10] G. Tapia et al., “Benchmarking YOLO models for intracranial hemorrhage detection using varied CT data sources,” *Computers in Biology and Medicine*, 2024.
- [11] T. H. Gencturk et al., “Detection and segmentation of subdural hemorrhage on head CT images,” *Medical Image Analysis*, 2024.
- [12] A. Pratama et al., “Metaheuristics approach for hyperparameter tuning of convolutional neural network,” *RESTI Journal*, 2024.
- [13] N. K. Kar et al., “Automated intracranial hemorrhage detection using deep learning in medical image analysis,” in *Proc. ICDSNS*, 2024.
- [14] M. Moazedi et al., “Deep learning methods for detecting and classifying brain hemorrhages with different preprocessing approach,” in *Proc. ICRoM*, 2024.
- [15] Y. Chen et al., “Automated intracranial hemorrhage detection using attention-based deep neural networks,” *Expert Systems with Applications*, 2024.
- [16] M. Hassan et al., “Deep ensemble learning for robust hemorrhage detection from CT images,” *IEEE Journal of Biomedical and Health Informatics*, 2024.

- [17] P. Singh et al., "Comparative analysis of deep learning models for brain hemorrhage detection," *Multimedia Tools and Applications*, 2024.
- [18] A. Verma et al., "CNN-based automated detection of intracranial hemorrhage using CT scan images," *Procedia Computer Science*, 2023.
- [19] S. Gupta et al., "Deep transfer learning for brain hemorrhage classification in medical imaging," *Healthcare Technology Letters*, 2023.
- [20] N. Ali et al., "Efficient deep learning model for early detection of intracranial hemorrhage," *Applied Sciences*, 2023.
- [21] S. Tanwar et al., "Advancing healthcare: CNN-based brain hemorrhage detection in intelligent environments," in *Proc. ICSD*, 2023.
- [22] D. Wang, R. Jin et al., "Real world validation of an AI-based CT hemorrhage detection tool," *Frontiers in Neurology*, vol. 14, 2023.
- [23] S. E. Arman et al., "Intracranial hemorrhage classification from CT scan using deep learning and Bayesian optimization," *IEEE Access*, 2023.
- [24] M. A. Mahjoubi et al., "Deep learning for cerebral hemorrhage detection and classification in head CT scans using CNN," in *Proc. IRASET*, 2023.
- [25] T. Badriyah et al., "Enhanced architecture of convolutional neural network (CNN) for intracranial hemorrhage prediction," in *Proc. ICSEC*, 2023.
- [26] M. Balipa and P. P. Kundapur, "Intracranial hemorrhage detection using deep learning and optimization techniques," in *Proc. ICUIS*, 2023.
- [27] O. Ozaltin, O. Yeniay, and A. Subasi, "Artificial intelligence-based brain hemorrhage detection," *Academic Press*, 2023.
- [28] A. Abdulkader et al., "Brain CT hemorrhage dataset," *Kaggle*, 2023.
- [29] A. Hussain et al., "An attention-based ResNet architecture for acute hemorrhage detection and classification," *IEEE Access*, vol. 10, 2022.
- [30] J. Y. Lee et al., "Detection and classification of intracranial haemorrhage on CT images using a novel deep-learning algorithm," *Scientific Reports*, 2020.



# LIST OF ENCLOSURES

## NPTEL Certificates



### NPTEL ONLINE CERTIFICATION

(Funded by the MoE, Govt. of India)

This certificate is awarded to  
**DOKKARI SOWJANYA**  
 for successfully completing the course

**Introduction to Machine Learning**

with a consolidated score of **54** %

|                    |          |                |       |
|--------------------|----------|----------------|-------|
| Online Assignments | 23.75/25 | Proctored Exam | 30/75 |
|--------------------|----------|----------------|-------|

Total number of candidates certified in this course: **9715**

**Jul-Sep 2025**  
(8 week course)






Indian Institute of Technology Kharagpur



Roll No: NPTEL25CS149S648200627

To verify the certificate 

No. of credits recommended: 2 or 3



### NPTEL ONLINE CERTIFICATION

(Funded by the MoE, Govt. of India)

This certificate is awarded to  
**TANKALA CHARISMA**  
 for successfully completing the course

**Introduction to Machine Learning**

with a consolidated score of **58** %

|                    |          |                |         |
|--------------------|----------|----------------|---------|
| Online Assignments | 23.54/25 | Proctored Exam | 34.7/75 |
|--------------------|----------|----------------|---------|

Total number of candidates certified in this course: **9715**

**Jul-Sep 2025**  
(8 week course)






Indian Institute of Technology Kharagpur



Roll No: NPTEL25CS149S648200174

To verify the certificate 

No. of credits recommended: 2 or 3



# NPTEL ONLINE CERTIFICATION

(Funded by the MoE, Govt. of India)

This certificate is awarded to

**KOMPALLI KEERTHI**

for successfully completing the course

## Introduction to Machine Learning

with a consolidated score of **56** %

|                    |          |                |          |
|--------------------|----------|----------------|----------|
| Online Assignments | 23.54/25 | Proctored Exam | 32.81/75 |
|--------------------|----------|----------------|----------|

Total number of candidates certified in this course: **9715**

Jul-Sep 2025

(8 week course)



**Skill India**

कोशल भारत - कुशल भारत



*Banerji*

**Prof. Haimanti Banerji**

Coordinator, NPTEL  
IIT Kharagpur



Indian Institute of Technology Kharagpur



**Elite**

# NPTEL ONLINE CERTIFICATION

(Funded by the MoE, Govt. of India)

This certificate is awarded to

**TEETI PRASAD**

for successfully completing the course

## Introduction to Machine Learning

with a consolidated score of **61** %

|                    |          |                |         |
|--------------------|----------|----------------|---------|
| Online Assignments | 23.54/25 | Proctored Exam | 37.5/75 |
|--------------------|----------|----------------|---------|

Total number of candidates certified in this course: **9715**

Jul-Sep 2025

(8 week course)



**Skill India**

कोशल भारत - कुशल भारत



*Banerji*

**Prof. Haimanti Banerji**

Coordinator, NPTEL  
IIT Kharagpur



Indian Institute of Technology Kharagpur



Roll No: NPTEL25CS149S548200297

To verify the certificate



No. of credits recommended: 2 or 3

# Paper Acceptance



## WAMS 2026 - Notification of acceptance - Oral Presentation - Paper ID: 437 – Reg

1 message

Microsoft CMT <noreply@msr-cmt.org>  
To: Tankala Charisma <charismatankala@gmail.com>

Sun, 1 Mar, 2026 at 11:44 am

Dear Dr/Prof Tankala Charisma,  
Thank you for your interest in WAMS 2026 and for submitting your paper.  
Congratulations! We are pleased to inform you that, based on the reviews received, your paper has been accepted for "Oral presentation" at the symposium. Please find the details below:

- Paper ID: 437
- Title: Metaheuristic-Assisted Deep Learning for Brain Hemorrhage Detection and Classification

We kindly request you revise the manuscript according to the review comments and ensure that the final Camera-Ready Paper maintains a similarity index below 40% (Including Bibliography) with no single source exceeding 5%.

The final Camera-Ready Copy (CRC) of the manuscript should be received on or before March 21, 2026. Strictly follow the CRC guidelines available through the following link while preparing your CRC.

<https://wams2026.com/crc-copyright-submission/>

You are encouraged to register to avail the early bird discounts. The registration details are available on the following web page

<https://wams2026.com/registration-2/>

Note: Only the accepted and registered papers (Both Oral and Poster) are submitted to IEEE Xplore for publication.

Looking forward to seeing you in WAMS-2026.

Best regards,  
TPC Chair, WAMS 2026

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our [Privacy Statement](#).

Microsoft Corporation  
One [Microsoft Way](#)  
[Redmond, WA 98052](#)



# Metaheuristic-Assisted Deep Learning for Brain Hemorrhage Detection and Classification

<sup>1</sup>Sreenubabu Dasari

Department of CSE

Aditya Institute of Technology and Management

Tekkali, Andhra Pradesh, India  
babu539@gmail.com

<sup>2</sup>Tankala Charisma\*

Department of CSE

Aditya Institute of Technology and Management

Tekkali, Andhra Pradesh, India  
charismatankala@gmail.com

<sup>3</sup>Dokkari Sowjanya

Department of CSE

Aditya Institute of Technology and Management

Tekkali, Andhra Pradesh, India  
sowjanyadokkari@gmail.com

<sup>4</sup>Kompalli Keerthi

Department of CSE

Aditya Institute of Technology and Management

Tekkali, Andhra Pradesh, India  
keerthikompalli890@gmail.com

<sup>5</sup>Teeti Prasad

Department of CSE

Aditya Institute of Technology and Management

Tekkali, Andhra Pradesh, India  
prasadteeti9@gmail.com

<sup>6</sup>Bodnaik Tharun Kumar

Department of CSE

Aditya Institute of Technology and Management

Tekkali, Andhra Pradesh, India  
tharunkumarbad3032@gmail.com

**Abstract**— Brain hemorrhage is a life-threatening condition for the nervous system and requires careful diagnosis. If not diagnosed properly, it can result in high mortality rates and complications. In recent years, improvements in deep learning have provided new methods for effectively analysing medical images. While Convolutional Neural Networks (CNNs) can perform medical analysis well, their effectiveness relies heavily on finely tuned parameters. This research project aims to create and apply an optimal deep learning solution for automatically classifying brain hemorrhage from medical images using a CNN and Particle Swarm Optimization (PSO). The study uses a CNN to analyse brain medical images for proper identification and classification. Furthermore, PSO can effectively identify and determine key parameters for deep learning algorithms. Extensive experimental investigations have proven the efficacy and effectiveness of the CNN-PSO framework, as it reaches high levels of classification accuracy, robustness, and dependability when it comes to the classification of various categories related to cases of brain hemorrhage. The experiments have also justified the significant improvement that can be ensured if Particle Swarm Optimization approaches are combined with Deep Learning algorithms for diagnosis purposes.

**Keywords**—CNN, Brain hemorrhage, Deep Learning, PSO algorithm

## I. INTRODUCTION

Brain disorders affect brain-related functions involving cognition, movement, speech, and memories. Another form of brain disorder is brain hemorrhage, which is a severe form of a disorder affecting the brain. It is usually caused by the bursting of blood vessels, resulting in bleeding thus leading to a rise in intracranial pressure and damage to neurons in the brain. Symptoms include a severe headache, weakness, difficulties in speech, or loss of consciousness [1].

Due to this fact, deep learning approaches are widely employed for efficient processing of vast biomedical data generated by IoMT technologies, since they automatically extract complex features and allow for fast analysis, thus enabling accurate, real-time clinical decision-making [2]. Complex image features can also be learned automatically by deep learning models without human intervention for feature engineering. Hierarchical image features extracted by CNNs can identify subtle image patterns corresponding to hemorrhage that can be used to successfully identify hemorrhage in CT scans. Among various deep learning models, CNN-based models have shown high performance for spatial feature extraction as well as the identification of

CT image slices as hemorrhage or non-hemorrhage to enhance the reliability of the process to identify hemorrhage [3]. With deep learning algorithms, intricate features of biomedical images can be learned by computers automatically without the need for human intervention in feature engineering. Convolutional Neural Networks (CNNs) pick out hierarchical image features that assist in identifying minute differences related to hemorrhage in CT scans to accurately detect hemorrhage in the brain. Within deep learning algorithms, high accuracy is achieved by CNN-based approaches to pick out spatial features as well as classify the CT scan into either hemorrhage or non-hemorrhage categories to increase [4]

Particle Swarm Optimization is a metaheuristic technique that is triggered by the social behavior of swarms and is known to work efficiently for hyperparameter optimization in deep learning networks. In medical imagery problems, studies using PSO have been found to optimize deep learning CNN parameters automatically, thus improving performance as opposed to doing the optimization manually [5].

## II. LITERATURE REVIEW

Dong-Wan Kang et al. [6] - 2025 – “Deep learning-assisted detection of Intracranial Hemorrhage” developed a deep learning system for detecting brain hemorrhage using CT images. Although the method achieved strong performance, it showed limitations in detecting small hemorrhages and specific subtypes, and its evaluation was limited to artifact-free CT scans.

Abhay Chaudhary et al. [7] - 2025 – “Deep learning models were employed for the classification of various types of hemorrhages within the cranial cavity from the CT scan images. But this technique faces the issue of high model complexity for combination. Although the model achieved high accuracy and introduced uncertainty-aware triage, its limitations include dependence on large annotated datasets, reduced generalization across diverse clinical settings, and challenges in detecting rare hemorrhage subtypes.

Tommaso D’Angelo et al. [8] - 2024 – “Accuracy and Time Efficiency of a Novel Deep Learning Algorithm for



Intracranial Hemorrhage Detection in CT Scans” proposed a deep learning technique for detecting a brain hemorrhage from non-contrast computed tomography scans. Despite the model's ability to accurately detect a brain hemorrhage with a faster diagnostic process, there were drawbacks, especially when dealing with a small or complicated case, since the study used retrospective data.

Gonzalo Tapia et al. [9] - 2024 - “Benchmarking YOLO Models for Intracranial Hemorrhage Detection Using Varied CT Data Sources” evaluated various YOLO models for the detection of intracranial hemorrhage in the CT scan images. The results indicated the effectiveness of the method for detecting hemorrhage accurately, but the drawbacks included the issue of class imbalance in medical imaging, low accuracy in less common hemorrhage classes like epidural hemorrhage, and the need for annotated data from sources like clinical trials.

Tugrul Hakan Gencturk et al. [10] - 2024 - “Detection and Segmentation of Subdural Hemorrhage on Head CT Images” presented a hybrid deep learning approach combining Mask Scoring R-CNN and Efficient Net for detecting and segmenting subdural hemorrhage from head CT scans. Although the method achieved high accuracy, its limitations include dependence on carefully annotated datasets, focus mainly on subdural hemorrhage rather than all hemorrhage types, and reduced generalization across diverse clinical imaging conditions.

## 2.1 Critical Analysis

Currently, almost all techniques developed to detect intracranial hemorrhage are based on the application of CNN-based techniques on the CT image; but their application is currently restricted to offline analysis with manually tuned parameters. The current application related to IoMT lacks optimization, thereby limiting its application in real-time analysis. The proposed study aims to apply the combination of CNN and PSO techniques on IoMT to optimize parameter tuning and facilitate real-time analysis. The proposed system is dependent on some factors related to the application of the system in intracranial hemorrhage identification within the IoMT setup. Factors in this case may include the quality of the images, hemorrhage size and type, as well as parameters involved in the implementation of the PSO algorithm.

## III. METHODOLOGY

- MRI – Provides clear and detailed brain images used as input data
- Deep Learning (DL) – Learns patterns from MRI images for automatic disease detection
- Convolutional Neural Network, CNN, is a deep learning method used to allow a computer to efficiently analyze images by learning and detecting meaningful patterns in images automatically.
- Particle Swarm Optimization (PSO) – Optimizes CNN settings to improve accuracy and reduce errors.

## 3.1 Convolutional Neural Network (CNN)

The method uses a CNN for classification of brain disorders using CT images. The preprocessing steps include resizing, normalization, removal of noises, and image augmentation. The network uses convolutional and pooling layers for feature extraction, and dense layers for classification with Softmax as the classifier. The network uses backpropagation for training, and the metrics used for evaluation are accuracy, precision, recall, and F1-score.

### 3.1.1 Convolutional Operation

$$Z_{i,j}^{(k)} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X_{i+m,j+n} W_{m,n}^{(k)} + b^{(k)}$$

$Z_{i,j}^{(k)}$  is Output feature map value  $M, N$  is Height and width of the filter  $i, j$  is Spatial position of output  $X_{i+m,j+n}$  is Pixel value at position  $W_{m,n}^{(k)}$  is Weight at kernel position  $(m, n)$ ,  $b^{(k)}$  is Bias of the  $k$ th filter.

### 3.1.2 Activation Function (ReLU)

$$P_{i,j} = \max_{(m,n) \in R_{i,j}} A_{m,n}$$

$A_{m,n}$  is Activated feature map values  $R_{i,j}$  is Pooling region (e.g.,  $2 \times 2$  window)  $P_{i,j}$  is Pooled output.

### 3.1.3 Global Average Pooling Layer

$$G_k = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W F_k(i, j)$$

$F_k(i, j)$  is activation at spatial location  $(i, j)$  of the  $k$ th feature map  $H$  is height of the feature map  $W$  is width of the feature map

### 3.1.4 Fully Connected Layer

$$y = f(\sum_{i=1}^n w_i x_i + b)$$

$x_i$  is Input feature  $w_i$  is Weight connected to input  $b$  is Bias  $n$  is Number of inputs  $f(\cdot)$  is Activation function  $y$  is Output neuron value

### 3.1.5 Sigmoid Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$x$  is Input value  $e$  is Euler's constant  $\sigma(x)$  is Probability output

### 3.1.6 Binary Cross-Entropy Loss

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

$N$  is Total samples  $y_i$  is True label (0 or 1)  $\hat{y}_i$  is Predicted probability  $\mathcal{L}$  is Loss value

## 3.2 Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) is used to optimize the parameters of the model by simulating the swarm intelligence concept. The parameters are adjusted by the swarm of particles, which represent the solutions to the optimization problem. PSO optimizes the significant hyperparameters of the CNN model, such as the learning rate, the number of filters, and the neuron size, based on the classification performance of the network, which improves the accuracy and optimizes the network.

Particle Swarm Optimization (PSO) is chosen because it is a simple and efficient optimization technique to tune hyperparameters of the neural network. PSO can efficiently optimize hyperparameters such as learning rate, batch size,



number of neurons, and dropout. PSO is simpler and more efficient compared to Bayesian Optimization, which can be applied to large search spaces.

### 3.2.1 Velocity Update

$v_t^{t+1} = wv_t^t + c_1r_1(pbest_t - x_t^t) + c_2r_2(gbest - x_t^t)$   
 $v_t^t$  is Velocity at iteration  $t$ ,  $w$  is Inertia weight  $c_1$  is Cognitive coefficient  $c_2$  is social coefficient  $r_1, r_2$  is Random values (0 – 1)  $pbest_t$  is Particle's best position  $gbest$  is Global best position  $x_t^t$  is Current position

### 3.2.2 Position Update

$x_t^{t+1} = x_t^t + v_t^{t+1}$   
 $x_t^t$  is Current position  $v_t^{t+1}$  is Updated velocity  $x_t^{t+1}$  is new position

### 3.2.3 Fitness Function

$f(x)$  = Validation Accuracy  
 $x$  is Hyper parameter set (learning rate, neurons, dropout)  
 $f(x)$  is CNN performance

### 3.2.4 Boundary Constraint

$x_t = \min(\max(x_t, x_{min}), x_{max})$   
 $x_{min}, x_{max}$  is Allowed bounds  $x_t$  is Particle position

## 3.3 CNN+PSO

The CNN-PSO method utilizes the CNN technique along with the Particle Swarm Optimization method, which helps in the efficient classification of hemorrhagic and non-hemorrhagic images using the CT scan technique. The pre-trained backbone used is the MobileNetV2 model, which helps in efficient feature extraction. The PSO method optimizes the learning rate, batch size, number of neurons, and dropout values by checking the accuracy of the model on the validation set.

The computational time of the CNN-PSO model is slightly higher compared to the CNN model. For the CNN model, training is done once. In contrast, for the PSO algorithm, parameters are optimized multiple times through multiple iterations. This process takes more time; however, it is helpful in tuning parameters for better brain hemorrhage detection classification.

### Algorithm

#### START

#### Declaration

$$CNN(x; \theta)$$

$$x_t^t, v_t^t, p_t^t, g^t$$

$$F(\cdot), \mathcal{L}, w, c_1, c_2$$

#### Input

$$D_{train} = \{(x_i, y_i)\}_{i=1}^N$$

$$D_{test}$$

#### Hyper Parameters

$$Image\ Size = (224 \times 224)$$

$$Batch\ Size = 32, Epochs = 30$$

$$X_i = [\eta_i, n_i, d_i]$$

$$\eta \in [10^{-5}, 10^{-3}]$$

$$n \in [64, 256]$$

$$d \in [0.3, 0.7]$$

$$w = 0.5, c_1 = 1.5, c_2 = 1.5$$

#### Dataset

$$D_{train}, D_{test} \leftarrow LoadImages(Directory)$$

$$D_{train} \leftarrow Shuffle(D_{train})$$

$$Image\ Resizing: x_i \leftarrow Resize(x_i, 224 \times 224)$$

$$Normalization: x_i^{norm} = x_i$$

$$Data\ Augmentation: x_i^{aug} = Zoom(Rotate(Flip(x_i^{norm})))$$

$$Prefetching: D_{train}, D_{test} \leftarrow Prefetch(D, AUTOTUNE)$$

#### Output

$$Optimized\ CNN\ Model\ CNN$$

#### CNN Training

$$z = GAP(f(x))$$

$$\hat{y} = \sigma(W_2 \phi(W_1 z))$$

$$\mathcal{L} = -1/N \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

#### PSO Initialization

$$x_i^0 \sim u(x_{min}, x_{max})$$

$$V_i^0 = 0, P_i^0 = x_i^0$$

$$g^0 = \arg \max F(P_i^0)$$

#### Fitness Evaluation

$$F(x_i) = Acc_{val}(CNN(x_i))$$

#### PSO Update Rule

For  $t=0 \rightarrow T-1$ :

$$v_t^{t+1} = wv_t^t + c_1r_1(p_t^t - x_t^t) + c_2r_2(g^t - x_t^t)$$

$$x_t^{t+1} = x_t^t + v_t^{t+1}$$

$$p_t^{t+1} = \arg \max \{F(p_t^t), F(x_t^{t+1})\}$$

$$g^{t+1} = \arg \max F(p_t^{t+1})$$

#### Final Model

$$x^* = g^T, CNN^* = CNN(x^*)$$

#### STOP

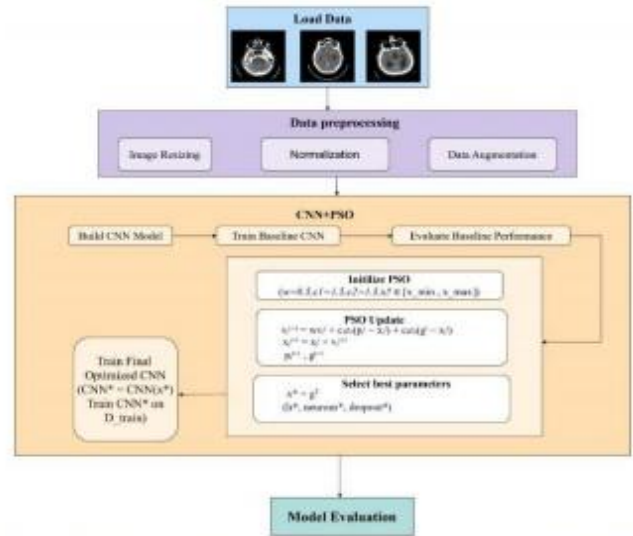


Figure 1: Above image illustrates the workflow of the proposed CNN-PSO architecture for brain hemorrhage detection. First, the brain images are loaded and preprocessed, followed by CNN training, then PSO optimization, and finally brain hemorrhage detection using the optimized CNN.

## IV. EXPERIMENTAL SETUP AND DATASET OVERVIEW

### 4.1 Experimental Setup

The experimental environment was developed using an HP Laptop 15-fc0xxx with AMD Ryzen 5 7520U processor, running at a speed of 2.80 GHz, along with 16 GB of RAM, AMD Radeon graphics, and 477 GB SSD storage, with Windows 11 Home (64-bit) operating system. The development of this research was performed in Python using Visual Studio Code as the development environment and



TensorFlow as the deep learning library of choice. Supporting libraries used in this research included NumPy, Pandas, Matplotlib, and Scikit-learn for data preprocessing and testing of the developed models. The system configuration was adequate for testing the developed deep learning model.

## 4.2 Dataset Overview

The “Brain CT Hemorrhage” dataset, available on the Kaggle platform, contains images related to the detection of hemorrhage using CT scan images. The images are classified as normal and hemorrhage cases, with the hemorrhage area indicated by high-density regions. The images are standardized in format and resolution as they are collected from real environments. Because of its simplicity and nature, the dataset is widely used to train deep learning models such as CNNs to classify brain conditions accurately.

## V. RESULT ANALYSIS

### 5.1 Convolutional Neural Network (CNN)

The proposed system has utilized two methods for brain hemorrhage detection from CT images using CNN. The first is a custom CNN, and the second is using transfer learning with MobileNetV2 for efficient brain hemorrhage detection from CT images. The MobileNetV2 model is implemented using TensorFlow and Keras, and it includes batch normalization and dropout for increased accuracy and efficiency of brain hemorrhage detection.



Figure 2: The graph highlights the variation of training and validation accuracy for each epoch during training. The similarity between the two graphs suggests a consistent training pattern and that this model can be relied upon for classification purposes.

In Figure 2, the CNN vs Validation Accuracy Curve after running the CNN model code on the VS Code platform. As the CNN is trained for varied epochs, the train accuracy rose from 65% accurate in the initial epochs to almost 94-95% in the final epochs, signifying effective feature extraction mechanisms within the CNN model’s convolutional layers. Conversely, the validation accuracy rose from 70% to 90-92%, signifying overall satisfactory performance in the validation datasets. A slight variability in the two accuracies is expected in the CNN model. This is due to the fact that the CNN model is being trained on the train datasets. A slight variability in the two accuracies signifies that there is minimal overfitting in the CNN model, which is an ideal sign in verifying that the CNN model is successfully executed on the VS Code platform, as evident.



Figure 3: This graph shows the training and validation loss of the proposed model across different epochs. The nearly similar loss values indicate balanced learning and stable model performance, which helps improve the overall classification results.

In Figure 3, Training & Validation Loss Curve obtained after 30 epochs. The training loss varied from around 0.7 in the early epochs to close to 0.25 in the latter epochs, whereas validation loss decreased from values around 0.6 to approximately 0.3. The very close behavior of both sequences proves sound completion of the learning procedure with little overfitting, hence successful CNN model training.

### 5.2 CNN+PSO

Particle Swarm Optimization (PSO) is used as a suitable hyperparameter tuning technique in the CNN model. Through the use of the NumPy and TensorFlow Keras libraries, PSO is used for iterations in obtaining optimal values of the respective parameters through modeling the social behaviors of the particles within the search space. In the current study, PSO optimizes critical parameters in the CNN model, such as learning rate, number of neurons, and dropout rate. PSO optimizes and adjusts these parameters in an intelligent manner such that faster convergent results and lower loss values are obtained for improved performance and increased robustness and generalization in the CNN model.

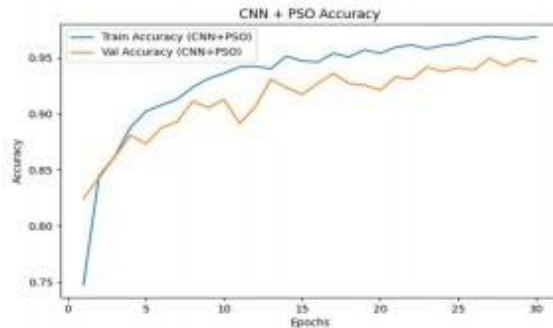


Figure 4: This graph shows the training and validation accuracy of the CNN + PSO model across different epochs. The nearly similar accuracy values indicate balanced learning and effective model performance, which helps improve the overall classification results.

In Figure 4, The accuracy of the CNN + PSO model during the training and validation phases for the 30 epochs. The accuracy in the train data goes from approximately 0.76 to 0.96, and the accuracy in the validation data goes from roughly about 0.78 to 0.93. The very close fit by both graphs implies that there is stable learning with very minimal occurrences of overfitting.



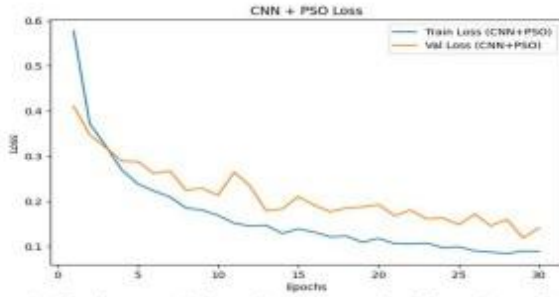


Figure 5: The above graph shows the training and validation loss of the CNN + PSO system. The loss values being almost equal express equally strong learning and optimal system performance, which assists in optimizing the overall classification outputs.

In Figure 5, The CNN+PSO learning and validation loss on 30 epochs. The loss falls from 0.6 to 0.15, and the validation loss improves from 0.45 to 0.2. That the learning and validation loss curves progress similarly indicates that the learning process is stable with less overfitting, ensuring successful optimization with CNN+PSO.

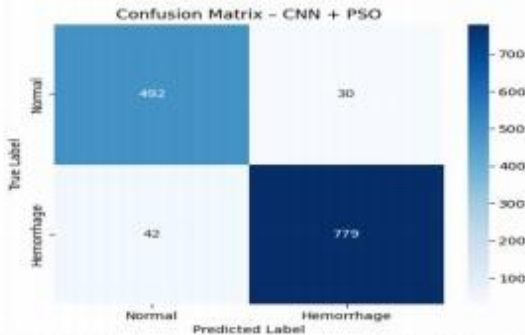


Figure 6: In this figure, the confusion matrix for the CNN + PSO model is shown. The classes are properly distinguished with high values in the diagonal, indicating that the classes are well classified, hence the overall performance of the model has been improved.

In Figure 6, The majority of the samples have been classified correctly by the classifier with 492 Normal images and 779 Hemorrhage images classified properly, while the incorrect classification samples-30 Normal images and 42 Hemorrhage images-indicate that classification with the help of PSO based optimization algorithm optimizes classification results and minimizes the incorrect classification of samples.

Test Accuracy: 0.9464

| Classification Report: |           |        |          |         |
|------------------------|-----------|--------|----------|---------|
|                        | precision | recall | f1-score | support |
| Normal                 | 0.92      | 0.94   | 0.93     | 522     |
| Hemorrhage             | 0.96      | 0.95   | 0.96     | 821     |
| accuracy               |           |        | 0.95     | 1343    |
| macro avg              | 0.94      | 0.95   | 0.94     | 1343    |
| weighted avg           | 0.95      | 0.95   | 0.95     | 1343    |

ROC-AUC Score: 0.9861

Model Evaluation Completed Successfully!

Figure 7: The following figure illustrates the classification performance of the CNN + PSO model. High accuracy and balanced precision, recall, and F1-score show that the classification is effective and the model performs reliably.

In Figure 7, Performance analysis of the CNN + PSO model shows that the model performs well with an overall test accuracy of 94.64 percent with well-balanced precision,

recall, and F1-score values for classes Normal and Hemorrhage. The high ROC-AUC value of 0.9861 indicates excellent separation between classes and overall performance.

The proposed system is based on MobileNetV2 as a backbone CNN model. MobileNetV2 is a lightweight CNN model. The trained model is stored using an HDF5 file format with an '.h5' extension. Hence, the model is of relatively smaller size compared to other deep learning models. Due to its lightweight nature, the trained CNN-PSO model can be implemented on normal computers, and GPUs can be utilized only to accelerate the training of the model.

## VI. CONCLUSION

A deep learning model is created that detects brain hemorrhage by using CNN with MobileNetV2 as the feature extractor. PSO is used to optimize the learning rate, batch size, number of neurons, and dropout value, hence improving the performance of the model. The model shows better performance when compared with the ResNet50 and VGG16 architectures. The model is useful in faster detection, which helps the doctor in emergency situations, though not recommended as the final solution, hence showing the effectiveness of the CNN-PSO model.

## VII. REFERENCES

- [1] S. Wright and S. Booth, "Brain Hemorrhage (Bleeding): Causes, Symptoms, Treatments," WebMD, Jun. 24, 2024.
- [2] P. Manickam et al., "Artificial Intelligence (AI) and Internet of Medical Things Assisted Biomedical Systems for Intelligent Healthcare," Biosensors, Jul. 25, 2022.
- [3] J. Y. Lee et al., "Detection and classification of intracranial haemorrhage on CT images using a novel deep-learning algorithm," Scientific Reports, Nov. 25, 2020.
- [4] A. Pratama et al., "Metaheuristics Approach for Hyperparameter Tuning of Convolutional Neural Network," RESTI J. Eng. Technol. Inf., Jul. 2024.
- [5] O. Ozbayoglu et al., "Design of an optimal convolutional neural network architecture for MRI brain tumor classification by exploiting particle swarm optimization," Remote Sensing, 2024.
- [6] D.-W. Kang et al., "Deep learning-assisted detection of intracranial hemorrhage," IEEE Access, 2025.
- [7] A. Chaudhary et al., "Multiclass intracranial hemorrhage detection and confidence-aware triage via deep ensemble learning," IEEE Transactions on Medical Imaging, 2025.
- [8] T. D'Angelo et al., "Accuracy and time efficiency of a novel deep learning algorithm for intracranial hemorrhage detection in CT scans," European Radiology, 2024.
- [9] G. Tapia et al., "Benchmarking YOLO models for intracranial hemorrhage detection using varied CT data sources," Computers in Biology and Medicine, 2024.ss
- [10] T. H. Gencturk et al., "Detection and segmentation of subdural hemorrhage on head CT images," Medical Image Analysis, 2024.
- [11] A. Abdulkader et al., "Brain CT Hemorrhage Dataset," Kaggle,